

1. INTRODUCTION

1.1 COMPANY PROFILE



Fig no: 1 Make Vision

MAKE Vision AI Software Solutions is a dynamic technology company based in Krishnagiri, dedicated to harnessing the power of Artificial Intelligence (AI) to drive innovation and transformation. We specialize in developing cutting-edge AI software solutions that empower businesses across industries to unlock new levels of efficiency, insight, and competitiveness.

Make Vision AI Software and Web Application Development Company was founded in 2021 with a dedicated and expert team of 15 members. The company's journey began as a response to the growing demand for cutting-edge AI solutions and innovative web applications in the digital landscape. With a vision of becoming a catalyst for positive change and driving technological advancements, the founders embarked on this entrepreneurial endeavor. The founding team brought together a diverse set of skills, including AI specialists, web developers, UX designers, and project managers, to create a well-rounded and dynamic workforce.

This commitment to innovation allowed Make Vision AI Software and Web Application Development Company to consistently deliver state-of-the-art solutions to its clients, ranging from small startups to large enterprises.

Our Vision

MAKE VISION is the way to revolutionize industries and improve quality of life through innovative AI solutions that drive efficiency, accuracy, and personalized experiences.

Our Mission

To be the global leader in AI innovation, transforming possibilities into realities and setting new standards for intelligence, automation, and human-machine collaboration.

Our Services

Our AI and software development services offer tailored solutions to meet the unique needs of each client, ensuring optimized performance, security, and scalability.

IT Outsourcing Services: You define your business environment and objectives. you can reduce costs, free up valuable resources, and concentrate on your core competencies while we handle what we excel at—IT services. As the leading IT outsourcing company in India, we provide solutions tailored to your software development needs, ensuring they are delivered on time and within budget.

Website Design: This is where our web design expertise and experience comes in to help you.

Grabbing your visitor's attention and keeping him interested on the site is what we do best!

Domain & Hosting services: Make Vision Analytics helps your business in suggesting the best domain names and hosting services which more affordable, speed, and reliable. we can provide exclusively low prices on the most popular domain names.

Search Engine Optimization: Be found by your customers at exactly the right moment. Maximize your online presence with professional SEO services from Make Vision Analytics. Find all the SEO marketing strategies and tactics you need to maximize your online presence and beat your competition with Make Vision Analytics. Optimize your website content to reach more customers through organic search.

Business App Development: With Make Vision Analytics services, you can not only create a remarkable app as unique as your business, but can also reduce the time and cost involved in hiring the resources or building a new solution for your needs.

Project Overview

Many bank clients will in general utilize **Automatic Teller Machines (ATMs)** and Internet entries to lead their **financial exchanges** with the mechanical advances in budgetary framework. Monetary clients particularly use ATMs for **physical exchanges** like **money withdrawal** or **money store**. Be that as it may, much the same as some other framework, ATMs are likewise experiencing various issues brought about by clients.

The principle objective of our work is to propose a PC vision structure which utilizes the installed ATM camera to perform **Face Recognition System (FRS)** so as to **forestall security breaks**. After the client embeds the card into the ATM, the proposed framework begins to perform **face recognition** and assembles a transitory face database for the client utilizing the **camera situated inside the ATM**.

In the event that the framework discovers there is an alternate client moving toward the ATM before the card holder, the ATM will send a caution at that point. This circumstance is on a very basic level not the same as biometric validation circumstances, in which a picture of an individual client is contrasted with an exhibition picture got from various conditions, perhaps some time before coordinating. The comparing picture and display picture right now all things considered isolated by mere minutes.

Develop a **robust security** model by combining physical access cards and electronic facial recognition technology to prevent unauthorized access and fraudulent activities at ATMs. Utilize **facial landmark algorithms** to accurately detect and authenticate users' faces, ensuring secure access to ATM services and **protecting account information**. Create a **user-friendly interface** with **intuitive navigation buttons**, allowing users to easily navigate through the **transaction process**, **enter their PIN**, and **complete transactions securely**.

Objective

- **Enhance Security with Facial Recognition:** Implement advanced facial recognition technology to authenticate users securely without requiring a physical card.
- **Eliminate Card Theft & Cloning Risks:** Prevent unauthorized access caused by lost, stolen, or cloned ATM cards by replacing traditional card-based authentication with biometric verification.
- **Enable Fast & Convenient Transactions:** Allow users to access ATM services quickly without needing to carry a physical card, reducing transaction time.
- **Reduce ATM Fraud & Unauthorized Access:** Use AI-powered liveness detection to prevent fraud attempts using photos, videos, or deep fake attacks.
- **Improve User Experience with Seamless Access:** Provide a hassle-free ATM experience with an intuitive interface, allowing users to authenticate and perform transactions with minimal effort.
- **Integrate with Mobile Banking Apps:** Enable users to pre-authorize withdrawals via their mobile banking app, using facial recognition as an extra layer of security.
- **Implement Multi-Factor Authentication (MFA):** Enhance security by requiring both facial recognition and PIN entry for high-value transactions.
- **Enable Secure Account Lock & Recovery Options:** Allow users to remotely lock their ATM access if suspicious activities are detected, ensuring account security.
- **Ensure End-to-End Encryption for Data Protection:** Secure biometric and transaction data using AES-256 encryption and secure communication protocols.
- **Implement Real-Time Alerts & Notifications:** Send instant alerts to users' mobile devices for every ATM transaction, allowing immediate fraud detection.
- **Comply with Financial Security Regulations:** Ensure compliance with GDPR, PCI DSS, ISO 27001, and banking cybersecurity standards to protect user privacy and financial data.

1.2 System Specification

1.2.1 Hardware Configuration

- Processor : INTEL I5 (7th generation)
- RAM : 8 GB RAM
- Hard disk : 64 BIT
- Monitor : 20' color monitor

Explanation:

Processor:

A processor, also known as the **Central Processing Unit (CPU)**, is the **brain of a computer**. It performs all calculations and operations required to run software programs and execute instructions. The **Intel Core i5 (7th generation)** is a mid-range processor from Intel's Kaby Lake family. It is known for its performance efficiency, power management, and support for modern applications.

RAM:

Random Access Memory (RAM) is a **temporary storage** that allows the computer to quickly access data and applications in use. More RAM means smoother multitasking and better system performance.

Hard Disk:

The given specification “**62 Bit**” seems to be a typo. It could refer to either **64-bit system architecture**, which allows handling more memory and performing complex operations. **Storage Device** (Hard Disk Drive or Solid-State Drive).

Monitor:

A monitor is an output device that displays visuals from the computer. The **20-inch color monitor** provides **clear images**, suitable for everyday tasks, gaming, and media consumption.

1.2.2 Software Configuration

- Front end : PYQT5
- Back end : Python
- Software used : PyCharm Community
- Platform : Windows 10

Front end:

PyQt5 is used for front-end development in this system. **PyQt5** is a **Python binding** for the **Qt framework**, one of the most widely used frameworks for creating graphical user interfaces (GUIs). PyQt5 allows developers to build **cross-platform desktop applications** with native look and feel, supporting not only Windows but also **macOS** and **Linux**.

PyQt5 is highly modular and provides developers with a rich set of **customizable widgets** such as **buttons**, **text boxes**, **sliders**, **menus**, and more making it ideal for creating sophisticated UIs. The flexibility of PyQt5 allows developers to design both simple and complex user interfaces that align with the needs of modern applications.

One of the key benefits of PyQt5 is its support for **multi-threading**, which allows background tasks to run independently of the main **UI thread**. This results in smoother applications that can handle **heavy computational workloads** or network requests without freezing the interface.

Furthermore, PyQt5 supports networking and **OpenGL integration**, which allows developers to create feature-rich applications, including real-time communication tools, **media players**, and **3D applications**. This makes PyQt5 a versatile framework that adapts to the diverse needs of developers.

PyQt5's flexibility also **extends to layout management**, enabling responsive design for applications that adjust to various screen sizes and resolutions. This capability is especially important for applications that need to be scalable across different devices and screen types.

The integration of Python with PyQt5 allows for quick development and prototyping of applications, enabling developers to iterate quickly and produce high-quality software.

Back end:

Python is chosen as the back-end language for its **simplicity**, **readability**, and **versatility**. Python is one of the most popular programming languages in the world, known for its clear and concise syntax that makes it easy for developers to **write**, **understand**, and **maintain code**. The ease of learning and use, combined with a strong focus on **readability**, makes Python a great choice for developers at all experience levels.

Python also has a vast ecosystem of libraries and frameworks that support various applications, from web development with **Flask and Django** to **scientific computing** with **NumPy** and **Pandas**. The rich collection of libraries allows developers to focus more on solving problems rather than reinventing the wheel. This language is widely used in backend services such as **APIs**, **databases**, and **application logic**, thanks to its ability to **quickly prototype** and deliver reliable systems. Python's support for asynchronous programming and concurrent execution also makes it well-suited for handling a large number of simultaneous requests, which is a common need for modern web applications.

Python's broad support for integration with other technologies, including databases, **cloud platforms**, and external APIs, makes it a strong choice for the back end. Python can easily interface with **SQL** and **NoSQL databases**, **manage connections** to external APIs, and interact with cloud storage solutions. Moreover, Python is **highly extensible** and has a large and **active community**, meaning developers can easily **find libraries**, resources, and support for nearly any task. The language's adaptability also allows for quick integration with machine learning models, which can be used in the back-end for data analysis or predictive tasks. applications and services.

Software used:

PyCharm Community Edition is the integrated development environment (IDE) used in this configuration. **PyCharm** is one of the most **popular IDEs for Python development** and offers a rich set of tools to streamline the development process. The Community Edition, which is free, includes features such as code completion, syntax highlighting, error detection, and a powerful debugger that helps developers identify and fix issues quickly. These features make it easier for developers to write error-free, efficient code.

PyCharm's ability to integrate with version control systems such as **Git** makes it a perfect choice for collaborative development. Version control helps developers **track changes**, **collaborate effectively**, and revert back to previous code versions when necessary. PyCharm also supports branching and merging, ensuring a smooth workflow for teams working on the same project.

In addition, PyCharm's support for virtual environments is a significant advantage. It allows developers to create isolated environments for each project, **preventing conflicts between libraries** and ensuring that the correct versions of dependencies are used. This is particularly important when working on multiple projects or collaborating with others.

The **IDE** also includes integrated testing tools that help developers write and run unit tests, ensuring the quality and reliability of the application. PyCharm simplifies testing by providing an interface to **run tests**, **view results**, and even **debug failing tests**. It supports various Python testing frameworks such as **unittest**, **pytest**, and **nose**, ensuring flexibility for developers.

Furthermore, PyCharm comes with integrated tools for code profiling, refactoring, and **documentation generation**, helping developers optimize their **code**, **improve performance**, and maintain high code quality throughout the project lifecycle.

Platform:

Windows 10 is the chosen operating system, providing a stable, secure, and user-friendly environment for software development. **Windows 10** is a **highly customizable operating system** that provides developers with the **flexibility** to adjust their workflow according to their preferences. The system supports Python and PyQt5 seamlessly, making it an **ideal platform for Python-based development**.

Windows 10 offers a wide range of tools and features that make it ideal for development, including built-in support for running Python scripts and applications. Developers can easily **install Python** and the **necessary development tools** on **Windows 10**, ensuring smooth and efficient software development. Virtual environments and package management can also be configured easily on Windows, ensuring a clean and organized workflow.

2. SYSTEM STUDY

2.1 EXISTING SYSTEM

2.1.1 Description

Human based security system, this provides **security** only to the **entrance door**. **Surveillance system** is too **difficult to detect unusual event** because it can be broken easily and also need large storage device to store every event and also thief may escape.

If some theft occurs, the system alerts only to police but arriving may take time. **Theft monitoring system in ATM** - In this security system, whenever **theft occurs**, the **ALARM rings** and hence the thief gets alerted and hence he may escape.

2.1.2 Drawbacks

- Relying solely on human presence for security is subject to **human vulnerabilities**.
- **Security personnel** may be distracted, **tired**, or influenced by external factors, potentially compromising the effectiveness of the security system.
- This manual process can be **time-consuming** and may **result in delays**, especially if security personnel are not in close proximity to the entrance door.

2.2 PROPOSED SYSTEM

2.2.1 Description

The ATM is equipped with a physical access card reader that allows users to insert their ATM cards to initiate transactions. An integrated **camera captures images** of individuals attempting to access the ATM. These images are then processed using **facial landmark algorithms** for **facial recognition**. The system employs facial landmark algorithms to **detect** and **extract unique features** from the facial images captured by the camera. These features are used to **authenticate users' identities**.

The ATM interface includes a **welcome screen** prompting users to **initiate transactions**. Users are guided through the **transaction process**, including **entering their PIN** and undergoing facial recognition authentication. Upon successful facial recognition authentication, the system verifies the **user's credit status** in real-time. A **database stores** the

facial images and associated user information for identification and authentication purposes. The database is accessed during the facial recognition process.

If an **authorized person** individuals **access to ATM services**, and **unauthorized person** there will be **mail alert** image capture to authorized person. If the **user** is **authenticated** and has sufficient funds, the **transaction proceeds**, and the **amount** is **printed in the console**. The system ensures that only authorized individuals with authenticated facial identities can access ATM services and perform transactions. **Unauthorized access** attempts are **detected** and **denied, enhancing** overall **security**.

2.2.2 Features

- The system offers dual authentication using physical access cards and facial recognition technology, ensuring robust security by requiring **multiple layers of verification** for user authentication.
- Utilizing a real-time credit verification system, the ATM ensures that only users with sufficient funds can proceed with transactions, **minimizing the risk of fraudulent activities**.
- The user interface is designed to be user-friendly, with **clear prompts** and **navigation buttons** guiding users through the transaction process, enhancing the overall user experience and accessibility.

2.3 SOFTWARE DESCRIPTION

The “**Cardless Transaction for Smart ATM Machine with Face Detection Security System**” is a cutting-edge banking solution that integrates advanced biometric authentication and **real-time transaction processing** to provide a seamless and highly secure ATM experience. This system eliminates the need for physical ATM cards by leveraging facial recognition technology, ensuring enhanced security and user convenience.

For the development of this smart ATM system, **Python** and **OpenCV** are utilized for face detection and recognition, while **Django/Flask** serves as backend framework for handling authentication and transaction processing. **PyCharm Community Edition** is used as the **Integrated Development Environment(IDE)** to coding, debugging, testing, and deployment.

Key Technologies and Features

1. Face Detection and Recognition

The system integrates **OpenCV**, **TensorFlow**, and deep learning algorithms to perform real-time facial recognition for user authentication.

Image Preprocessing: Captures and processes the user's face image to ensure accurate identification.

Feature Extraction: Uses deep learning models (such as CNNs) to analyze unique facial features and match them with the stored database.

Anti-Spoofing Mechanism: Detects fake images or videos to prevent unauthorized access.

2. Secure Authentication Process

The ATM employs **Multi-Factor Authentication (MFA)** to enhance security. The user initiates a transaction by entering their registered phone number or scanning a QR code. The system scans the user's face and matches it with the registered biometric data. An OTP (One-Time Password) is sent to the user's mobile phone for final authentication.

3. Real-Time Transaction Processing

To ensure seamless transactions, the system integrates banking APIs for real-time fund verification and money transfers.

Instant Balance Verification: Confirms account balance before processing a withdrawal.

Quick Cash Dispensation: The ATM releases cash instantly upon successful authentication.

Transaction Logs: Stores transaction details securely for future reference and auditing.

3. SYSTEM DESIGN AND DEVELOPMENT

3.1 FILE DESIGN

The file design for the proposed system aiming to **secure online transactions** through **facial recognition technology** employs a structured approach for efficient organization and management of its components. At the core lies the main program file, serving as the central point of interaction and coordination among various modules.

This file orchestrates user inputs, system responses, and the execution flow of the application. Supporting the main program file are several essential components, including the **Database Management File** responsible for handling **user data storage** and **retrieval**, the **Facial Recognition Module** integrating algorithms for **facial landmark detection** and **comparison**, and the **OTP Generation and Validation File** ensuring secure transaction authentication.

Additionally, the system incorporates User Interface Files for presenting intuitive interfaces to users, a Logging and Error Handling File for tracking system activities and handling exceptions, a Configuration File storing application settings, and a Testing and Evaluation File containing scripts and routines for comprehensive system evaluation.

Through this organized file design, the system promotes **code maintainability**, **scalability**, and **collaboration** among developers, facilitating efficient development and deployment of a **robust online transaction** security solution. The ATM security system using facial recognition follows a **structured file design** for efficient organization and functionality.

At its core, the main program file orchestrates **user interactions**, **system logic flow**, and necessary module calls for authentication and transaction processing. Supporting this, the Database Management File securely **stores user credentials**, **facial images**, and **transaction history** while managing data retrieval and encryption.

The Facial Recognition Module implements face detection algorithms, processes facial features, and compares stored images with **real-time captures** for authentication. Additionally, an OTP Generation and Validation File enhances security by generating and

verifying one-time passwords via email or SMS. The User Interface Files ensure an intuitive experience by providing screens for login, transactions, and authentication steps. To maintain security and system integrity, the Logging and Error Handling File records user activity, authentication attempts, and transaction logs while managing error detection and resolution.

The Configuration File defines system settings, authentication thresholds, and security policies, allowing **easy modification** of preferences. Lastly, the Testing and Evaluation File includes scripts for testing system components, **validating facial recognition accuracy**, and ensuring overall performance. The structured file design ensures seamless operation and coordination between various modules, making the system highly efficient in mitigating security risks.

The Main Program File acts as a bridge, enabling **smooth communication** between different components and ensuring secure transactions. The Database Management File plays a crucial role in storing encrypted user data, providing a secure repository that facilitates **quick retrieval** and **verification**. The Facial Recognition Module integrates **cutting-edge AI** and **machine learning techniques** to enhance facial landmark detection and matching, making authentication more precise and robust.

The **OTP Generation** and Validation File add an extra security layer by **sending a dynamic password via SMS or email** to the user's registered contact information. This mechanism ensures that even if an **unauthorized person** gains access to a user's ATM card, they **cannot proceed without the one-time password**. The User Interface Files contribute significantly to user satisfaction by providing a seamless and visually appealing experience, guiding users through the authentication process with clarity and simplicity.

The Logging and Error Handling File is indispensable in tracking user activities, logging successful and unsuccessful authentication attempts, and identifying potential security breaches. This file ensures system stability by providing logs that aid in debugging and refining the system to enhance security and performance. Similarly, the Configuration File offers flexibility in modifying system preferences, **authentication criteria**, and **transaction limits**, allowing administrators to **fine-tune security measures** as needed.

Finally, the Testing and Evaluation File ensures the system's reliability and robustness by running automated tests, verifying accuracy rates, and assessing system efficiency. Continuous evaluation helps in **identifying vulnerabilities**, **refining algorithms**, and ensuring seamless user experience while maintaining top-notch security standards. By following this comprehensive and structured file design, the ATM security system leveraging facial recognition technology offers a secure, **scalable**, and **efficient solution** to prevent unauthorized access and **fraudulent transactions**.

3.2 INPUT DESIGN

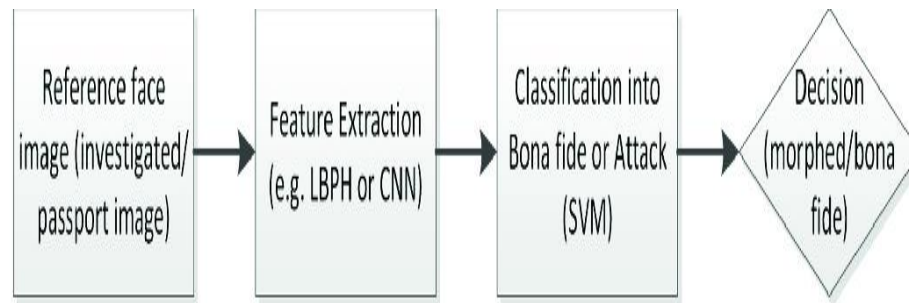
3.2.1 Facial Landmark Algorithm

Facial landmarks is a technique which can be applied to applications like **face alignment**, **head pose estimation**, **face swapping**, **blink detection**, **drowsiness detection**, etc. In this context of facial landmarks, our vital aim is to **detect facial structures** on the person's face using a method called **shape prediction**.

Facial Landmarks Detection has 2 steps:

1. To detect the **key facial structures** on the person's faced.
2. It involves **localizing the face** in the image.

We can do Face detection in a number of ways. We can use **OpenVMS** built-in **Hear Cascade XML files** or even **Tensor Flow** or using **Keras**. Over here especially, we need to apply a **HOG (Histogram of Gradients)** and Linear **SVM (Support Vector Machines)** object detector specifically for the task of face detection. We can also do it using **Deep Learning-based algorithms** which are built for **face localization**. Also, the algorithm will be used for the detection of the faces in the image.



A) Morphing detection without a paired probe image.

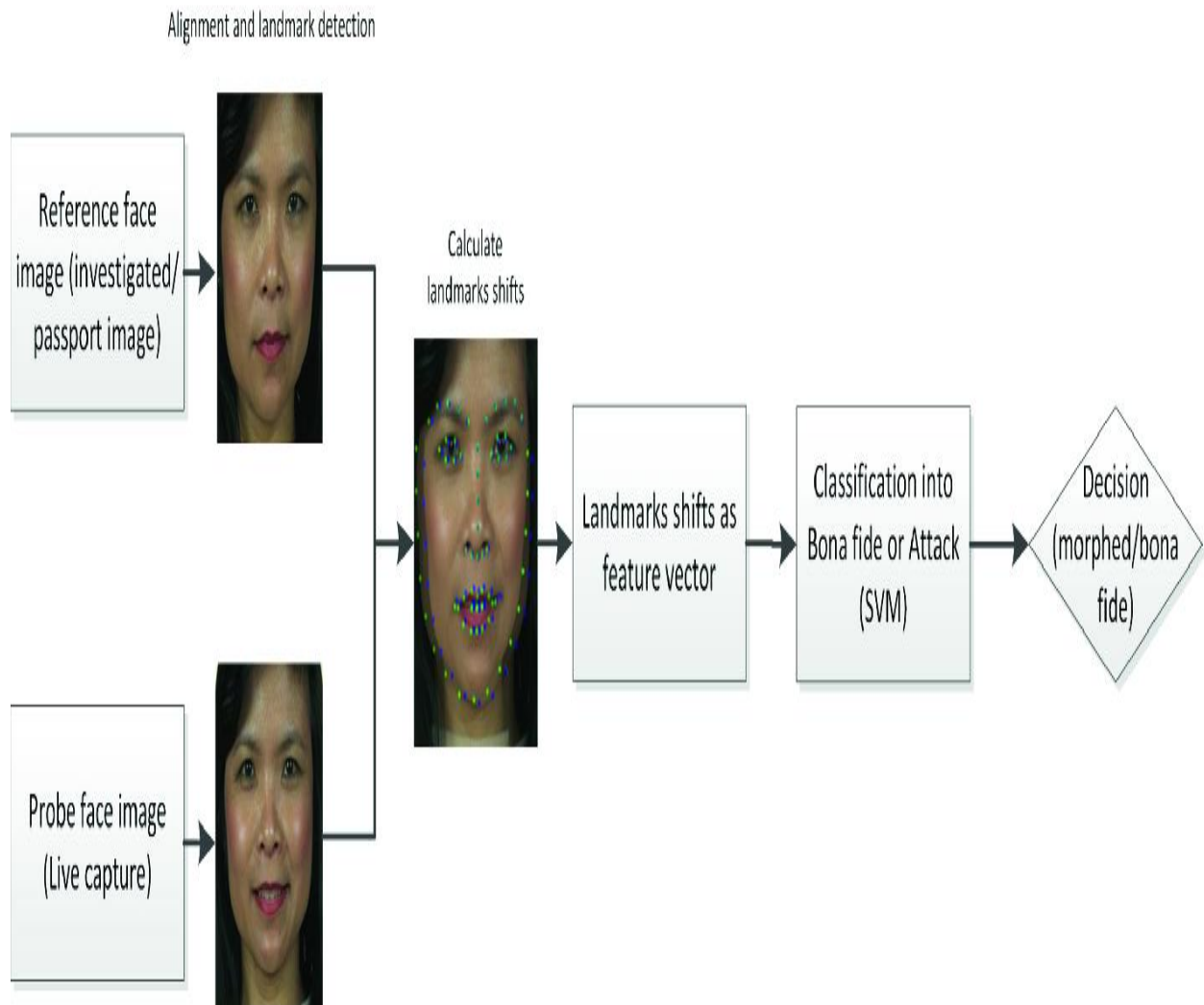


Fig no: 3.1 Face Detection Using Facial Landmark

1. D-Face Recognition Using LBP

Local Binary Pattern (LBP) Is an easy, but very strong user of texture who **marks** the **image's pixels** with each **pixel's proximity**, and views the outcome as binary numbers. It was first defined in **1994 (LBP)** and is a **powerful component** for the **classification of texture** since then.

In addition, LBP has been calculated to increase significantly the detection efficiency on certain datasets when coupled with histograms of oriented gradients (HOG) descriptor. By using **LBP combined with histogram**, face pictures with a simple data vector can be described.

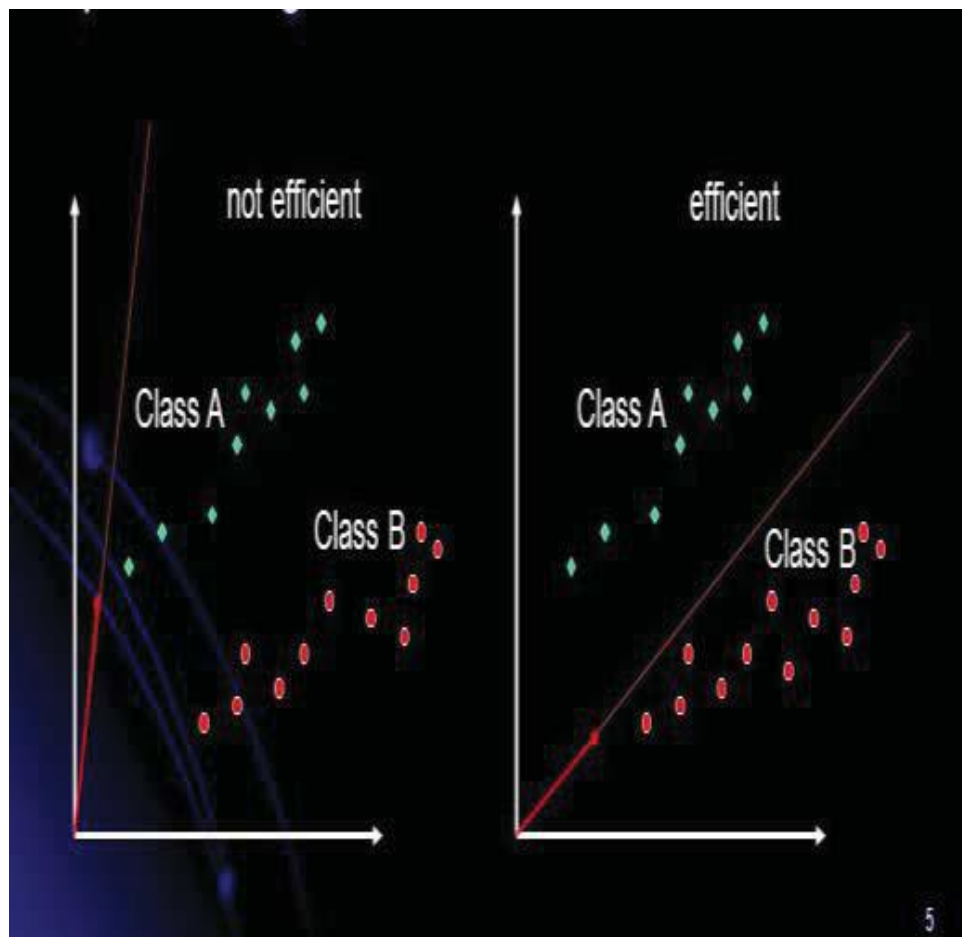


Fig no: 3.2 PCA seeks directions that are efficient for representing the data.

Local Binary Patterns, or LBPs for short, the work popular et al as **texture descriptor**, **Multi resolution Grayscale** and **Rotation Invariant**.

Texture Classification with Local Binary Patterns (the definition of **LBP**s was implemented at the beginning of **1993**). It computes a global representation of texture based on the **Gray Level Co-occurrence Matrix**, unlike Hara lick. LBP's compute a regional texture representation. The binary image is built by comparing every pixel with the pixel surrounding area .For example, look at the original **3 x 3-pixel** neighborhood LBP descriptor operated just same this:

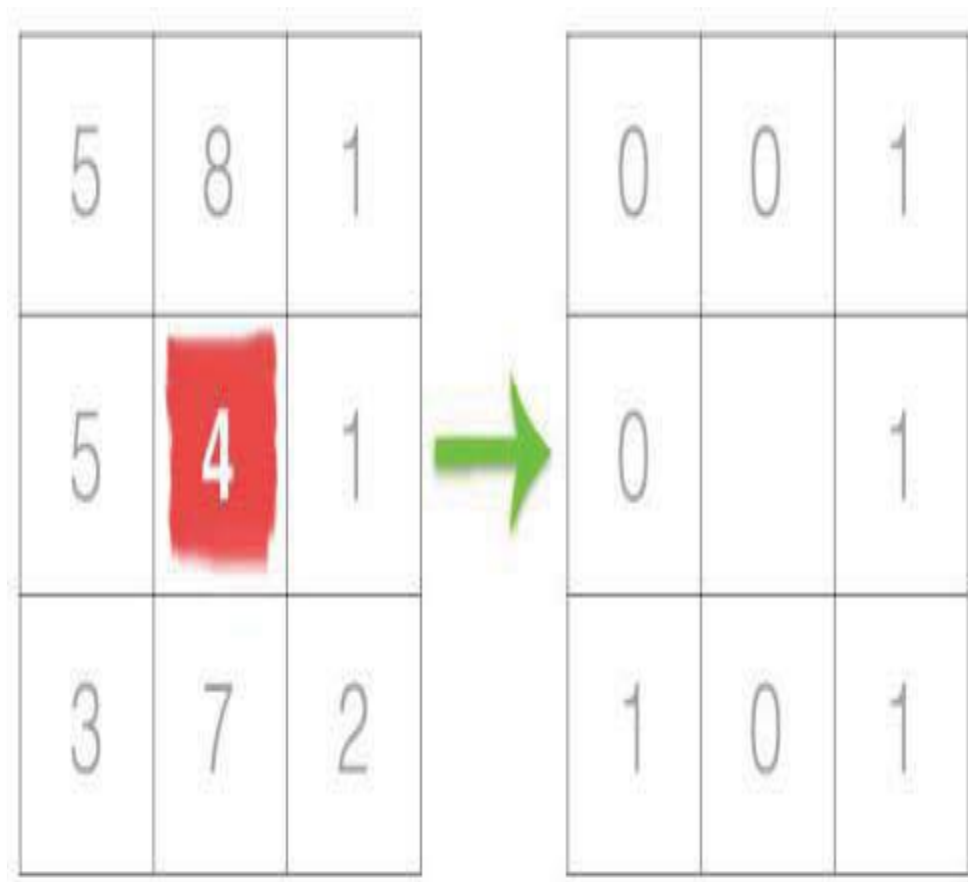


Fig no: 3.3 Binary Pattern

A middle pixel and an **8-binary numbers cap** is the first step in creating an LBP for the **8 Pixels neighborhood**. In the figure above, we choose the center pixel and set it against its 8pixel neighborhood. If the **pixel center is more than** the following line, **set the value to 1**; **set the value to 0**, otherwise. A total combination of $2^8 = 256$ possible LBP codes with eight pixels will be achieved. Thanks to a neighborhood of 3x3, we have 8 neighbors on which we have to check binary. This binary search results were stored in an **8-bit array** and **converted into decimal**.

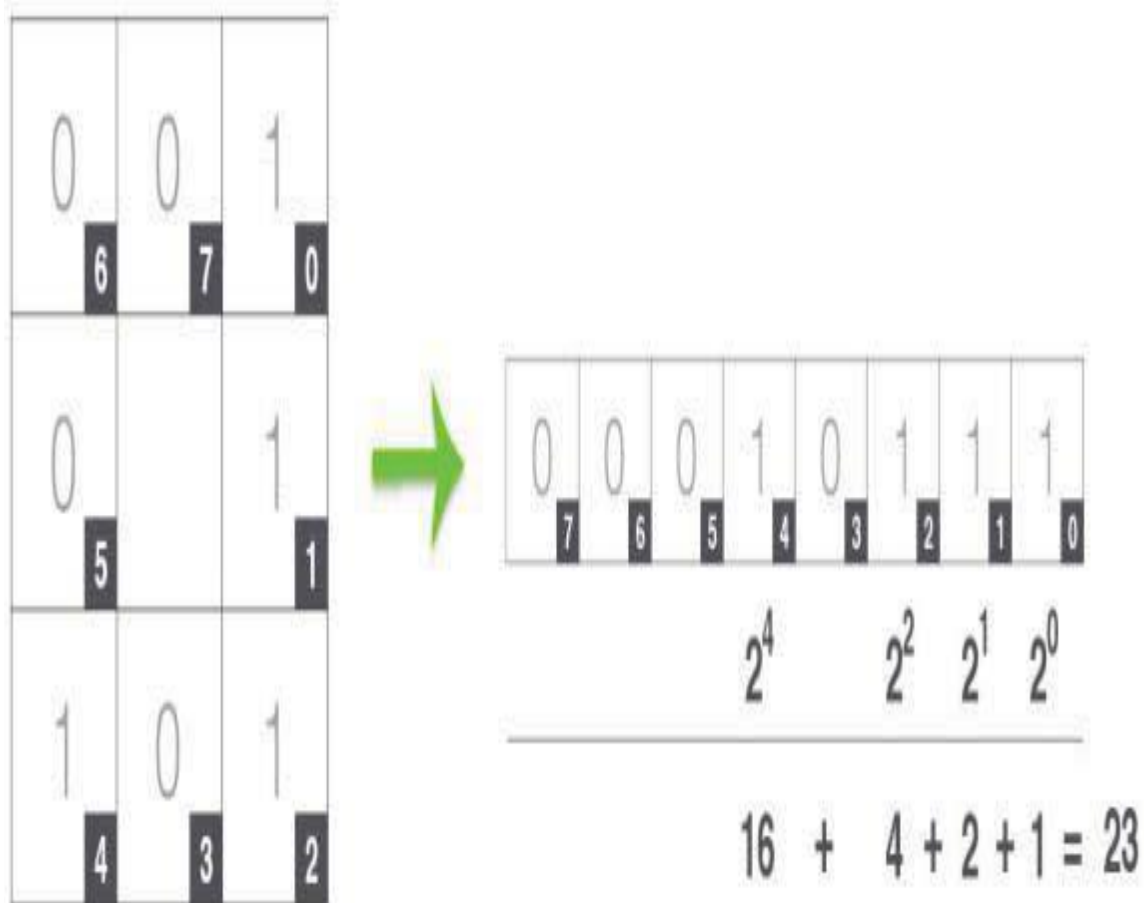


Fig no: 3.4 Face Recognition of LBPH Algorithm

3.2.2 Segmentation Process

Segmentation is the **separation of one or more regions** or **objects** in an image based on a discontinuity or a similarity criterion. A region in an image can be defined by its border (edge) or its interior, and the two representations are equal.

1. Region-Based Segmentation

In this segmentation, we grow regions by **recursively including** the **neighboring pixels** that are **similar** and **connected** to the **seed pixel**. We use similarity measures such as differences in gray levels for regions with homogeneous gray levels. We use connectivity to prevent connecting different parts of the image.

2. Top-down approach

First, we need to define the **predefined seed pixel**. Either we can define all pixels as seed pixels or **randomly chosen pixels**. Grow regions until all pixels in the image belongs to the region.

3. Bottom-Up approach

Select seed only from objects of interest. Grow regions only if the similarity criterion is fulfilled.

4. Similarity Measures:

Similarity measures can be of different types: For the grayscale image the similarity measure can be the different textures and other **spatial properties, intensity difference** within a region or the **distance between mean value** of the region.

5. Region merging techniques:

In the region merging technique, we try to combine the regions that contain the **single object** and **separate it** from the background. There are many regions merging techniques such as **Watershed algorithm, Split and merge algorithm**, etc.

6. Pros:

Since it performs simple threshold calculation, it is **faster to perform**. Region-based segmentation works better when the object and background have **high contrast**.

7. Image Segmentation

Image segmentation is a commonly used technique in **digital image processing** and analysis to **partition an image into multiple parts** or regions, often based on the characteristics of the pixels in the image. Image segmentation could involve separating foreground from background, or clustering regions of pixels based on **similarities in color** or **shape**.

Image Segmentation mainly used in

- Medical imaging
- Face detection
- Machine vision
- Autonomous Driving

There are two types of image segmentation techniques.

1. Non-contextual thresholding : Thresholding is the simplest non-contextual segmentation technique. With a **single threshold**, it transforms a greyscale or **color image into a binary image** considered as a binary region map. The binary map contains two possibly disjoint regions, one of them containing **pixels with input data values** smaller than a threshold and another relating to the input values that are at or above the threshold. The below are the types of thresholding techniques

- i. Simple thresholding
- ii. Adaptive thresholding
- iii. Color thresholding

2. Contextual segmentation: Non-contextual thresholding groups pixels with no account of their relative locations in the image plane. Contextual segmentation can be more successful in separating individual objects because it accounts for **closeness of pixels** that belong to an individual object. Two basic approaches to contextual segmentation are based on signal discontinuity or similarity. Discontinuity-based techniques attempt to find complete boundaries enclosing relatively uniform regions assuming abrupt **signal changes** across each boundary. Similarity-based techniques attempt to directly create these uniform regions by grouping together **connected pixels** that satisfy certain similarity criteria. Both the approaches mirror each other, in the sense that a complete boundary **splits one region into two**. The below are the types of Contextual segmentation.

- i. Pixel connectivity
- ii. Region similarity
- iii. Region growing
- iv. Split-and-merge segmentation

3. Texture Segmentation: Texture is most important attribute in many **image analysis** or **computer vision applications**. The procedures developed for texture problem can be subdivided into four categories.

- i. Structural approach
- ii. Statistical approach
- iii. Model based approach
- iv. Filter based approach

3.2.3 Image Processing

As a Machine Learning Engineer, **data pre-processing** or data cleansing is a crucial step and most of the ML engineers spend a good amount of time in data pre-processing before building the model. Some examples for data pre-processing includes **outlier detection**, **missing value treatments** and **remove** the unwanted or **noisy data**.

Similarly, Image pre-processing is the term for operations on images at the lowest level of abstraction. These operations do not increase image information content but they decrease it if entropy is an information measure. The aim of pre-processing is an improvement of the **image data** that **suppresses undesired distortions** or enhances some image features relevant for further processing and analysis task.

There are 4 different types of Image Pre-Processing techniques and they are listed below.

- i. Pixel brightness transformations/ Brightness corrections
- ii. Geometric Transformations
- iii. Image Filtering and Segmentation
- iv. Fourier transform and Image restoration

Let's discuss each type in detail.

1. Pixel Brightness Transformations)PBT(

Brightness transformations **modify pixel brightness** and the transformation depends on the properties of a pixel itself. In PBT, **output pixel's value** depends only on the corresponding **input pixel value**. Examples of such operators include brightness and **contrast adjustments** as well as **color correction** and **transformations**.

Contrast enhancement is an important area in image processing for both human and computer vision. It is widely used for medical image processing and as a pre-processing step in **speech recognition, texture synthesis, and many other image/video processing applications.**

There are two types of Brightness transformations and they are below.

- i. Brightness corrections
- ii. Gray scale transformation

The most common Pixel brightness transforms operations are

- i. Gamma correction or Power Law Transform
- ii. Sigmoid stretching
- iii. Histogram equalization

Two commonly used point processes are multiplication and addition with a constant.

$$G)x(=\alpha f)x(+\beta$$

The parameters $\alpha > 0$ and β are called the **gain** and **bias parameters** and sometimes these parameters are said to **control contrast and brightness** respectively.

cv.convertScaleAbs (image, alpha=alpha, beta=beta(

For different values of alpha and beta, the image brightness and contrast varies.

3.2.4 Key Point Descriptor

A **SIFT descriptor** of a local region (key point) is a **3-D spatial histogram** of the **image gradients**. The gradient at each pixel is regarded as a sample of a three-dimensional elementary feature vector, formed by the pixel location and the gradient orientation.

Binary image descriptors **encode patch appearance** using a **compact binary string**. The hamming distance in this space is designed to follow a desired image similarity measure typically sought to be invariant to scene illumination and viewpoint changes.

A **feature descriptor** is an **algorithm** which takes an **image** and **outputs feature descriptors/feature vectors**. Feature descriptors encode interesting information into a series

of numbers and act as a sort of numerical "**fingerprint**" that can be used to **differentiate one feature** from another.

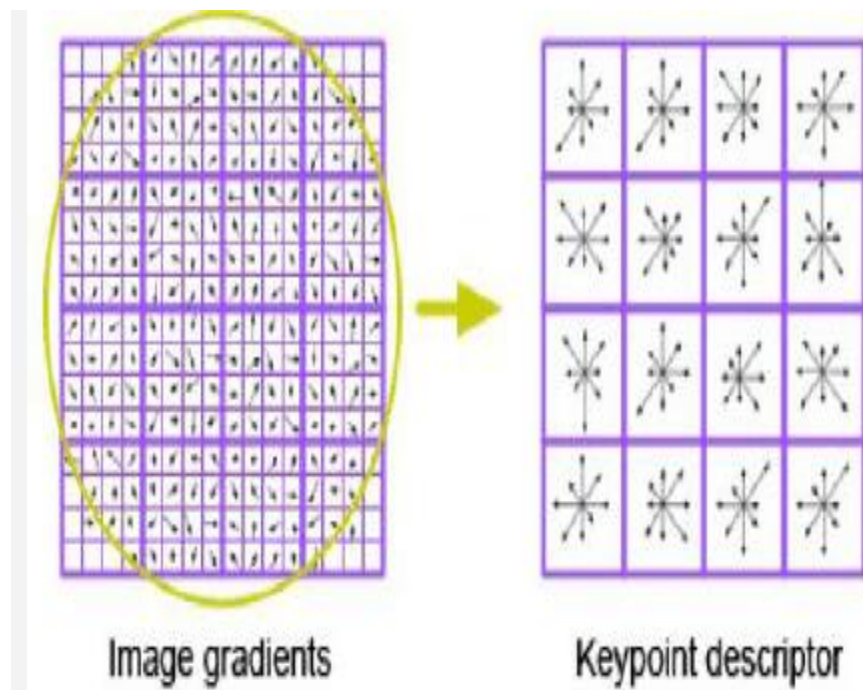


Fig no: 3.5 Key Point Descriptor

3.2.5 Morphology Operation

Morphological Operations

A morphological operation is a set of **image processing algorithms** that acts on **image pixels** using **pre-defined kernels**. These kernels, known as **structuring elements**, define patterns that are used to process images. Bellow, you can see some examples of common structuring elements.

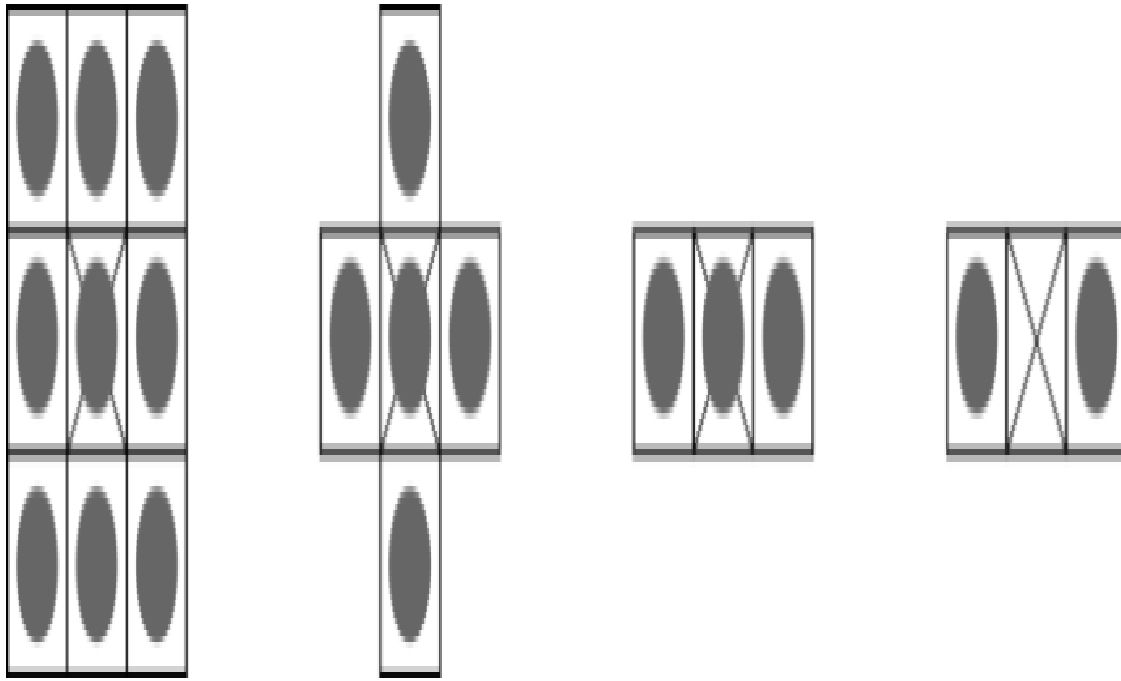


Fig no: 3.6 Morphology Operation

You can imagine these elements as simple matrices. Here, **black circles** represent the **value 1** (foreground). Also, structuring elements are defined based on some local origin (marked as Xs). The most common morphological operations are **dilation and erosion**.

In both methods, a kernel convolves an image and substitute the target (origin) pixel following a criterion. Intuitively, erosion has the effect of deteriorating the boundary pixels of foreground elements. We can think of it as **expanding the background pixels** and **reducing the shape of foreground elements**.

In short, a pre-defined kernel is passed over the image as a **2D convolution**. Then, assuming a binary image, an **input pixel** will be **1** if all the pixels under the **kernel is 1**. Otherwise, it gets a **0**. We can also think of erosion as applying a minimum function over the

pixels under the kernel. In this way, the output pixel will be the smallest value among all the pixels under the kernel. As we will see shortly, erosion has also the effect of **removing white noise**.

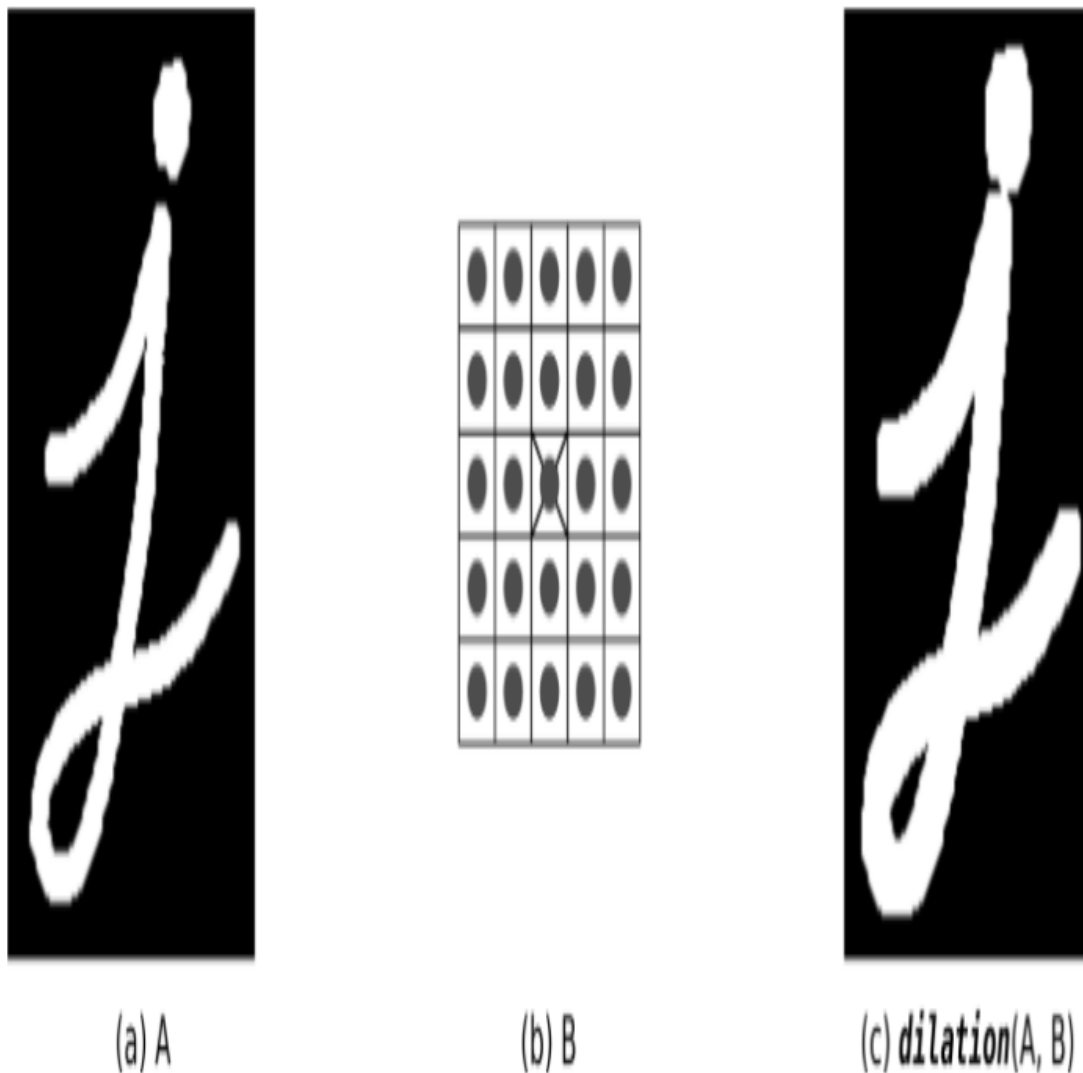


Fig no: 3.7 Dilation Process

Dilation acts in the opposite way. It **expands the boundaries of foreground elements**. As a consequence, it also **reduces the number of background pixels**. As oppose to **erosion**,

given an **input pixel**, the **output is 1** if at least **one** of the **input pixels** (under the kernel) is **1**. Otherwise, the **output is 0**.

Similarly, dilation applies a maximum operations over the pixels under the kernel. In other words, the output pixel will be the maximum value among all pixels that fall within the kernel.

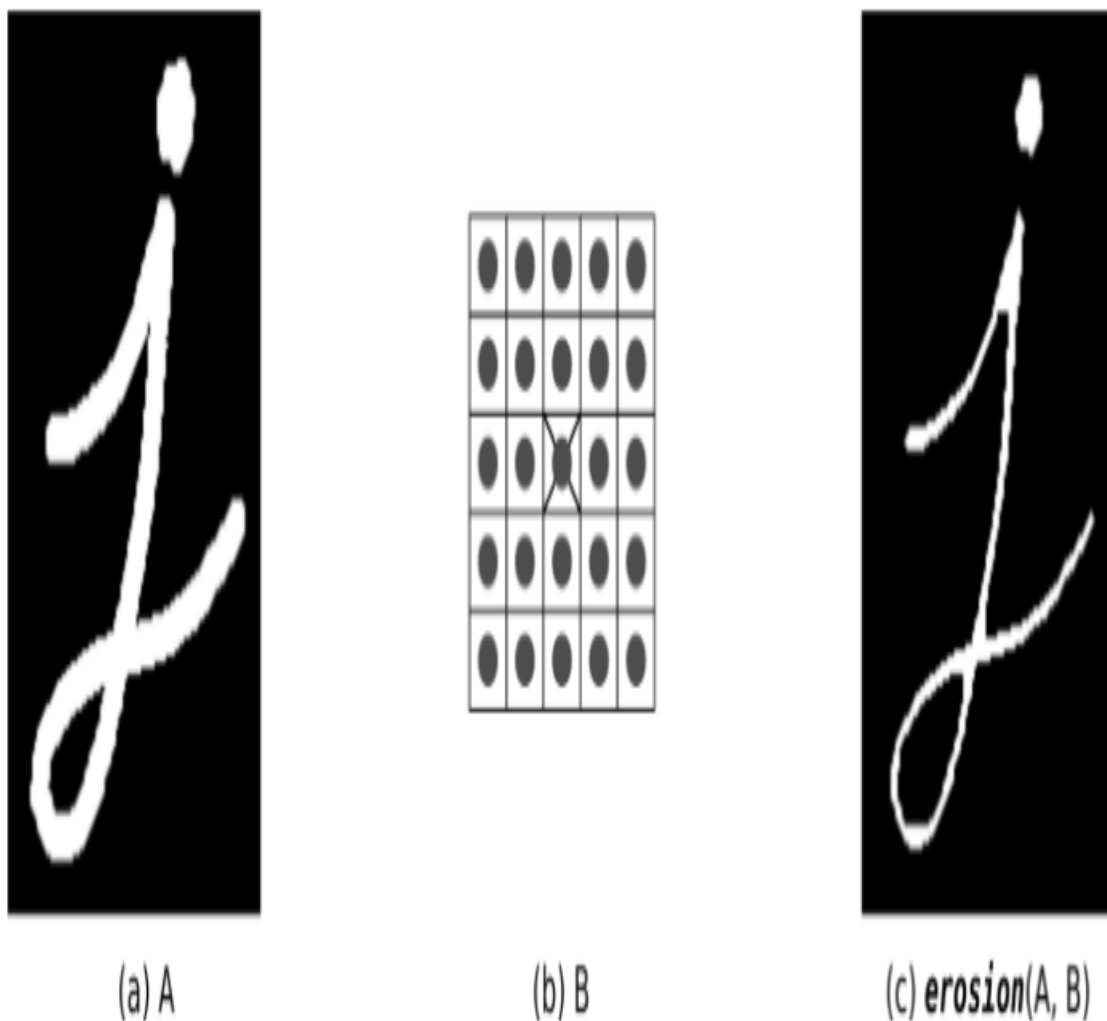


Fig no: 3.8 Erosion Process

3.2.7 Mediapipe

Mediapipe is a comprehensive **open-source library developed by Google** that provides a flexible framework for **building multi-modal machine learning (ML)** pipelines. Specifically designed for handling perception tasks, Mediapipe offers **pre-built solutions** for various applications, including **face detection** and **facial landmark tracking**, which aligns with the objectives of the proposed ATM Security System based on Face Recognition, PIN, and OTP. Within the context of this project, Mediapipe can enhance the face recognition component by efficiently handling facial landmarks, **tracking facial features**, and ensuring the robustness of the authentication process.

Utilizing Mediapipe's facial landmark tracking capabilities, the system can **accurately locate and analyze key points** on a **user's face**, allowing for a more precise and reliable face recognition process. This not only improves the security of the ATM system but also **enhances** the overall **user experience** by providing a seamless and efficient means of authentication. Integrating Mediapipe into the ATM Security System enables advanced facial landmark tracking and analysis, contributing to the system's ability to accurately verify the user's identity through face recognition. This integration enhances the overall security of the proposed authentication system, providing a reliable and user-friendly solution for accessing ATM services **without the need** for traditional **ATM cards** and **PINs**.

3.2.8 Dlib's 68 Face Algorithm

- Depending on the **facial appearance** and **facial shape patterns**, Facial landmark detection algorithms are classified into three. The landmark detection is carried out in two steps.

Face detection:

- Face detection locate a human face from the image and return a value in terms of the co-ordinates of the bounding rectangle of the detected face.

Landmarks detection:

- After detecting the face, we have go through points inside the bounding rectangle for key points.

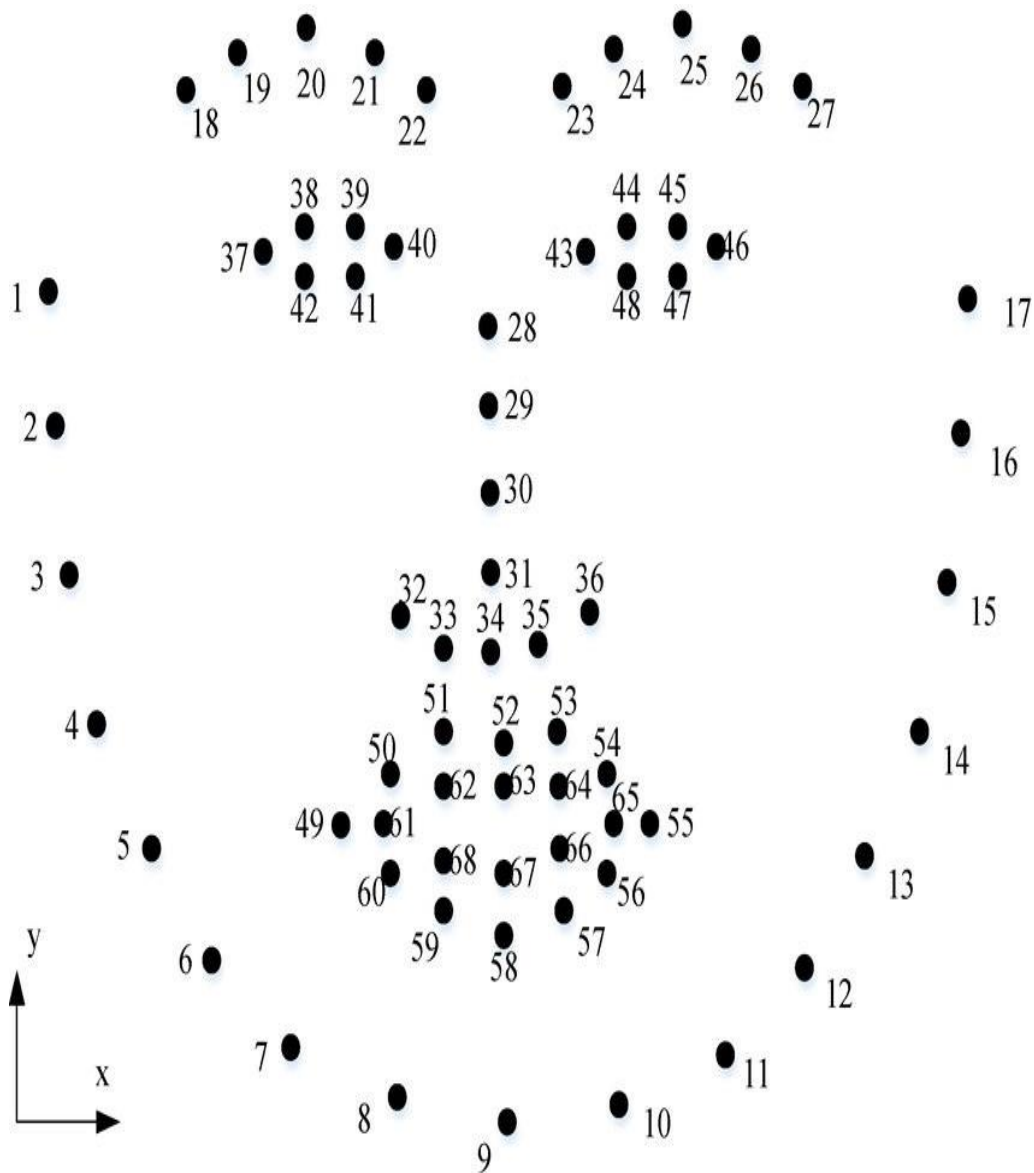


Fig no: 3.9 Dlib'S 68 Face Algorithm

Dlib's 68 Face Features:

The below image is an example of a **Dlib's 68 points model**. There we can see those points from 1 to 68. But sometimes we don't need all 68 feature points, then for that, we will do in the next post, how we can customize those points according to our requirements. In this post, we only going to see about 68 Dlib does points for clear understand.

Facial landmark points detection through Dlib's 68 Model:

There are mostly two steps to detect face landmarks in an image which are given below:

1. **Face detection:** Face detection is the first methods which locate a human face and return a value in x, y, w, h which is a rectangle.
2. **Face landmark:** After getting the location of a face in an image, then we have to through points inside of that rectangle.

There are many methods of face detector but we focus in this post only one which is Dlib's method. Like, Opencv uses methods **LBP** cascades and **HAAR** and Dlib's use methods **HOG (Histogram of Oriented Gradients)** and **SVM (Support Vector Machine)**.

1. Facial landmark detection algorithm

Facial landmark detection algorithms help to automatically identify the locations of the facial key landmark points on a facial image or from a video. The key landmark points normally include the **facial regions** like **nose tip**, **eye corner**, **eye brows** and **chin tip**. Some applications of facial landmark detection are **face swap**, **head pose detection**, **detecting facial gestures**, **gaze direction** etc.

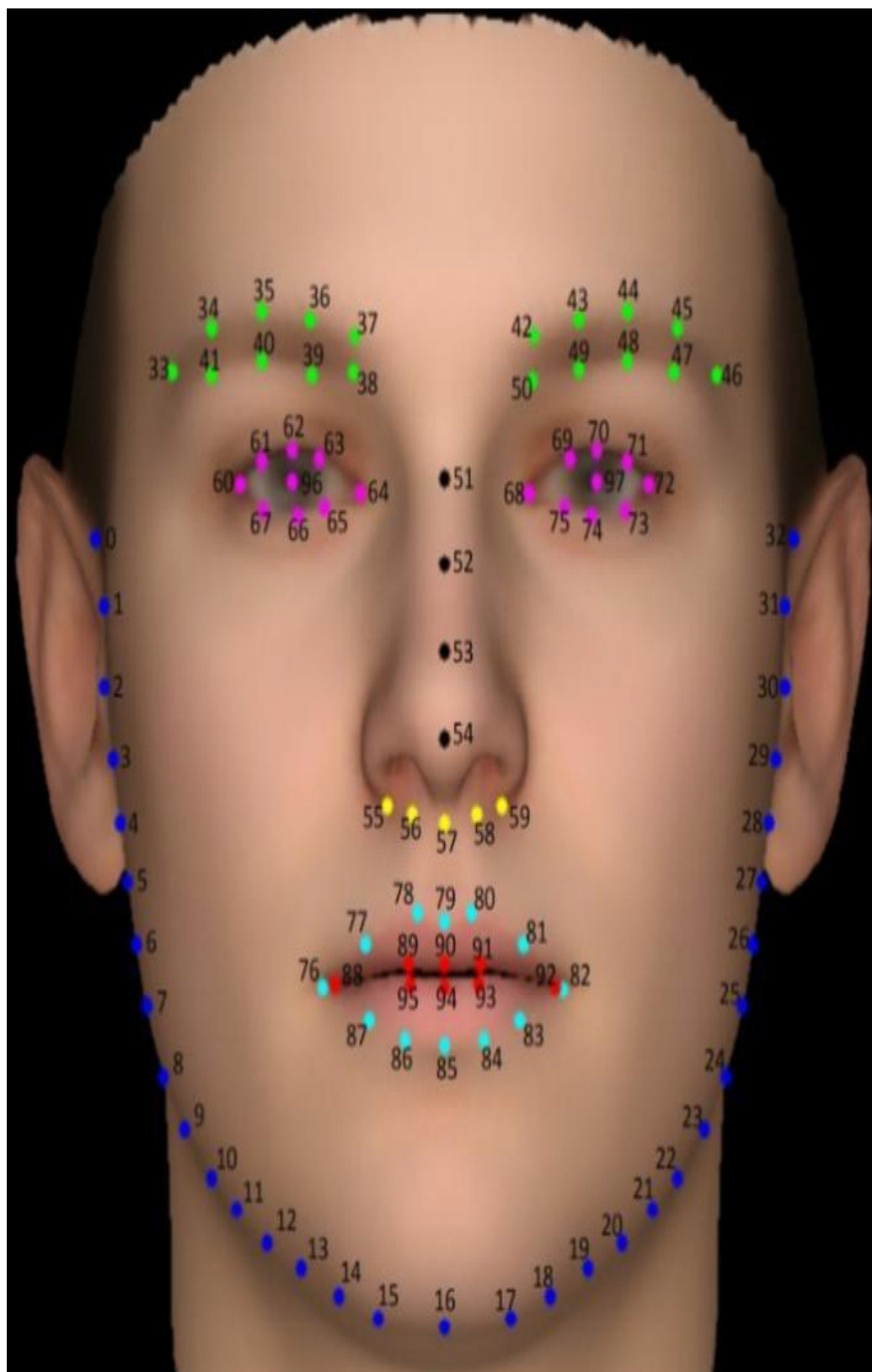


Fig no: 3.10 Marking Face Landmark

The landmark detection is carried out in two steps.

1. **Face detection**

Face detection locate a human face from the image and return a value in terms of the co-ordinates of the bounding rectangle of the detected face.

2. **Landmarks detection**

After detecting the face, we have go through points inside the bounding rectangle for key points.

Classification of Algorithms

Depending on the **facial appearance** and **facial shape patterns**, Facial landmark detection algorithms are classified into three.

Here, the facial appearance refers to the distinctive pixel intensity patterns around the facial landmarks or in the **whole face region**, while **face shape patterns** refer to the patterns of the face shapes as defined by the landmark locations and their spatial relationships.

The three major categories of landmark detection algorithms are:

- a. Holistic methods
- b. Constrained Local Model (CLM) methods
- c. Regression-based methods

The holistic methods explicitly model the holistic facial appearance and global facial shape patterns. CLMs rely on the explicit local facial appearance and explicit global facial shape patterns.

The regression-based methods use holistic or local appearance information and they may embed the **global facial shape** patterns implicitly for joint landmark detection.

Some recent methods combine the **deep learning models** and **global 3D shape** models for landmark detection.

PyCharm introduction

PyCharm is the most popular **IDE** used for **Python scripting language**. This chapter will give you an introduction to PyCharm and explains its features.

PyCharm offers some of the best features to its users and developers in the following aspects

1. Code completion and inspection
2. Advanced debugging
3. Support for web programming and frameworks such as Django and

Flask

Features of PyCharm

Besides, a developer will find PyCharm comfortable to work with because of the features mentioned below

Code Completion

PyCharm enables **smoother code completion** whether it is for built in or for an external package.

SQL Alchemy as Debugger

You can set a breakpoint, pause in the debugger and can see the SQL representation of the user expression for **SQL Language code**.

Git Visualization in Editor

When coding in Python, queries are normal for a developer. You can check the last commit easily in PyCharm as it has the blue sections that can define the difference between the last commit and the current one.

Code Coverage in Editor

You can run **.py** files outside PyCharm Editor as well marking it as code coverage details elsewhere in the project tree, in the summary section etc.

Package Management

All the installed packages are displayed with proper **visual representation**. This includes list of installed packages and the ability to search and add new packages.

Local History

Local History is always keeping **track** of the **changes** in a way that complements like Git. Local history in PyCharm gives complete details of what is needed to rollback and what is to be added.

Refactoring

Refactoring is the process of **renaming one or more files** at a time and PyCharm includes various shortcuts for a smooth refactoring process.

User Interface of PyCharm Editor

The user interface of PyCharm editor is shown in the screenshot given below. Observe that the editor includes various features to create a new project or import from an existing project.



Fig no: 3.11 PyCharm Software

PyCharm installation

From the screenshot shown above, you can see the newly created project Demo and the site-packages folder for package management along with various other folders.

In this chapter, you will learn in detail about the installation process of PyCharm on your local computer.

Steps Involved

You will have to follow the steps given below to install PyCharm on your system. These steps show the **installation procedure** starting from downloading the PyCharm package from its official website to creating a new project.

Step 1

Download the required package or executable from the official website of PyCharm: <https://www.jetbrains.com/pycharm/download/#section=windows>. Here, you will observe two versions of the package for Windows as shown in the screenshot given below. The two versions are the **Professional Edition** and the **Community Edition**.

On the other hand, the **Community Edition** is free and open-source, suitable for basic Python development. Choose the edition based on your needs and preferences.

If you're just starting or working on personal projects, the Community Edition is generally sufficient. After selecting the appropriate version, click the **Download** button to start the installation process.



Fig no: 3.12 PyCharm Installation

Note that the professional package involves all the advanced features and comes with free trial for few days and the user has to buy a licensed key for activation beyond the trial period. Community package is for free and can be downloaded and installed as and when required. It includes all the basic features needed for installation. Note that we will continue with community package throughout this tutorial.

Step 2

Download the **community package (executable file)** onto your system and mention a destination folder as shown below –



Fig no: 3.13 PyCharm Community Package

Step 3

Now, begin the installation procedure similar to any other software package.

Step 4

Once the installation is successful, PyCharm asks you to **import settings of the existing package if any.**



Fig no:3.14 PyCharm Package

This helps in creating a new project of Python where you can work from the scratch. Note that unlike other IDEs, PyCharm only focusses on working with projects of Python scripting language.

This chapter will discuss the basics of PyCharm and make you feel comfortable to begin working in PyCharm editor.

When you launch PyCharm for the first time, you can see a welcome screen with entry points to IDE such as

- Creating or opening the project
- Checking out the project from version control
- Viewing the documentation
- Configuring the IDE

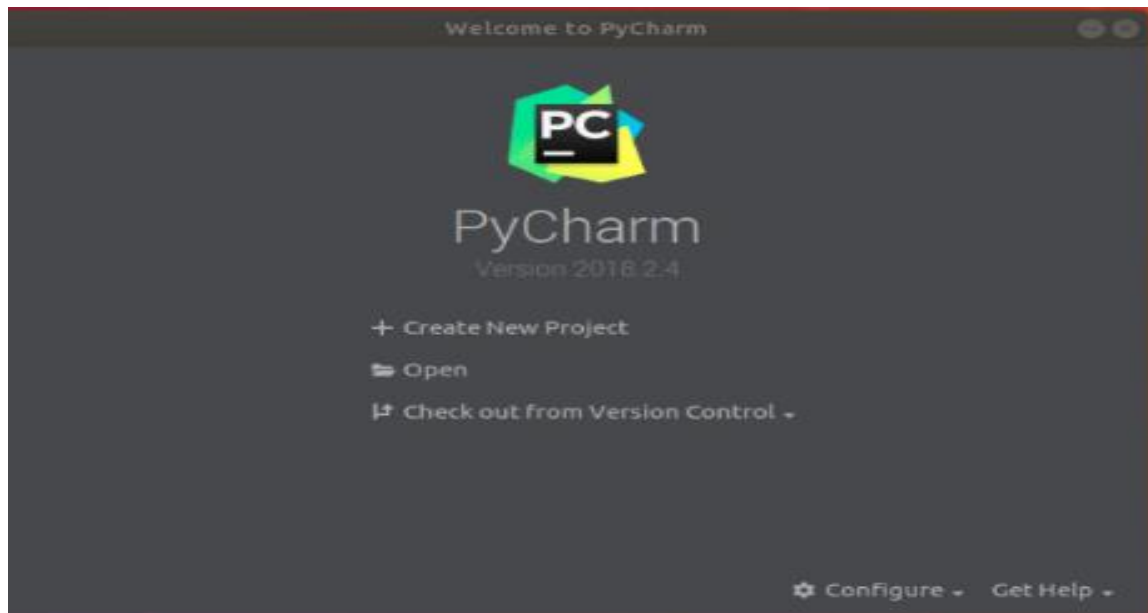


Fig no: 3.15 PyCharm Configure Page

Recall that in the last chapter, we created a project named demo1 and we will be referring to the same project throughout this tutorial.

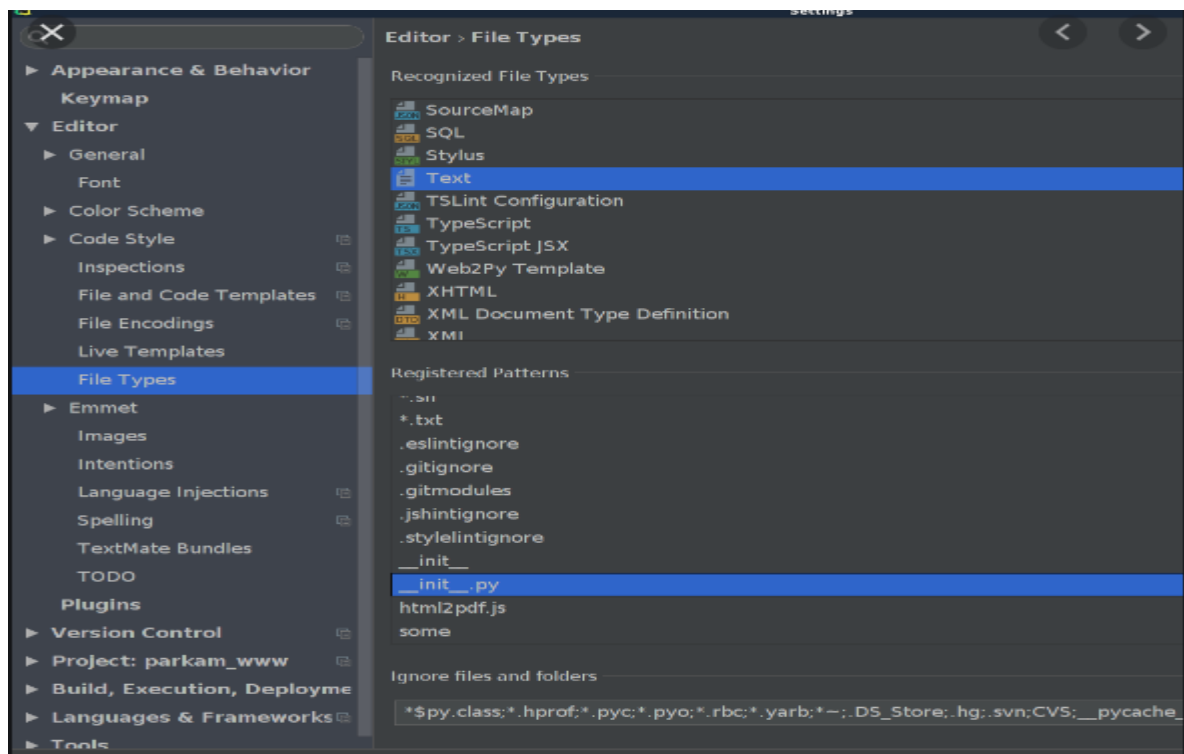


Fig no: 3.16 Editing Page

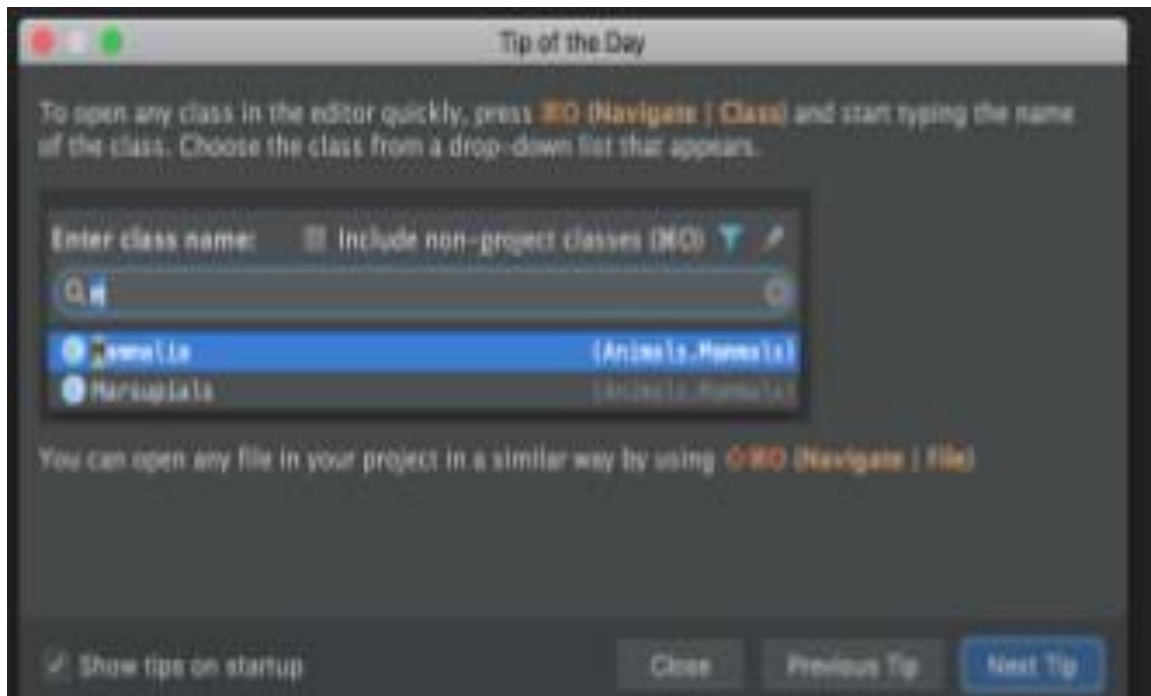


Fig no: 3.17 Editor Page

The above snapshot describes the project overview of demo1 and the options to create new file. Let us create a new file called main.py.

Once the file is created, it will be added to the project directory, allowing you to begin writing the code. In **the PyCharm IDE**, you can simply right-click on the project folder and choose **New > Python File to create the file**.

After creating the file, you can start adding the Python code that will be executed when running the program. The code included in main.py is as follows:

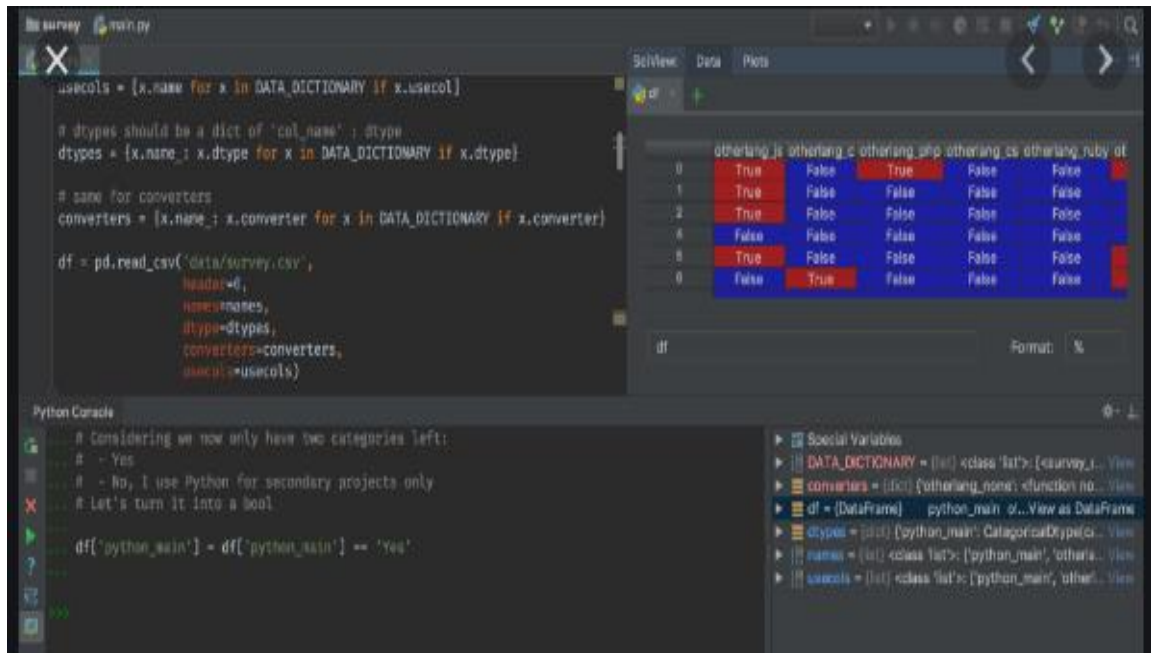


Fig no: 3.18 Coding Window

The code created in the **file main.py** using PyCharm Editor is displayed as shown below This code can be run within IDE environment. The basic demonstration of running a program is discussed below

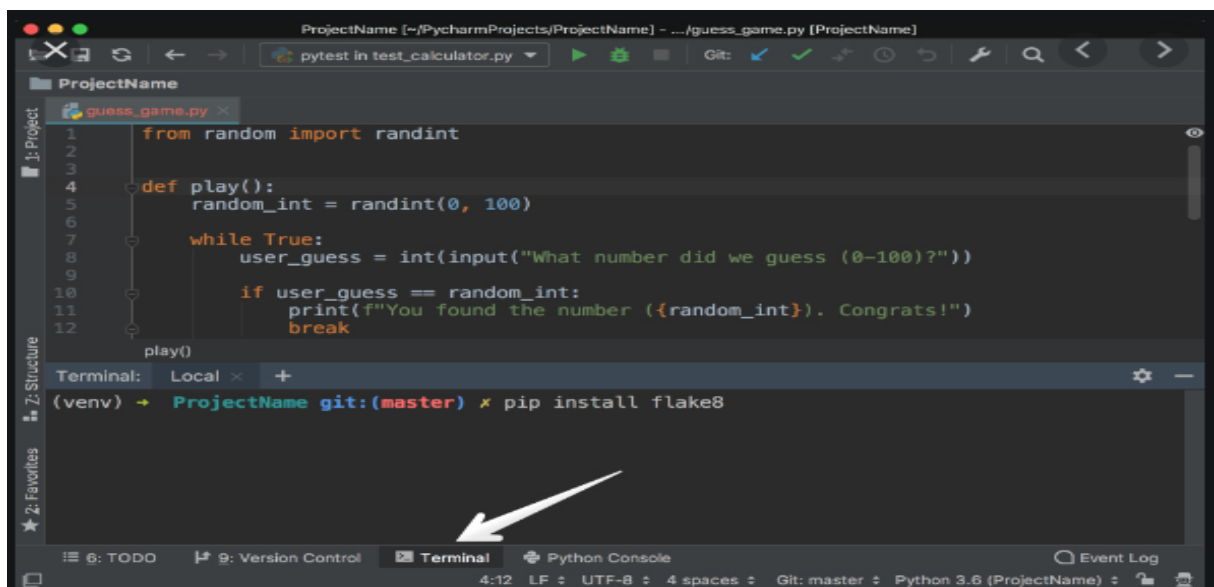


Fig no: 3.19 Coding Window Page

Note that we have included some errors within the specified code such that console can execute the code and display output as the way it is intended to.

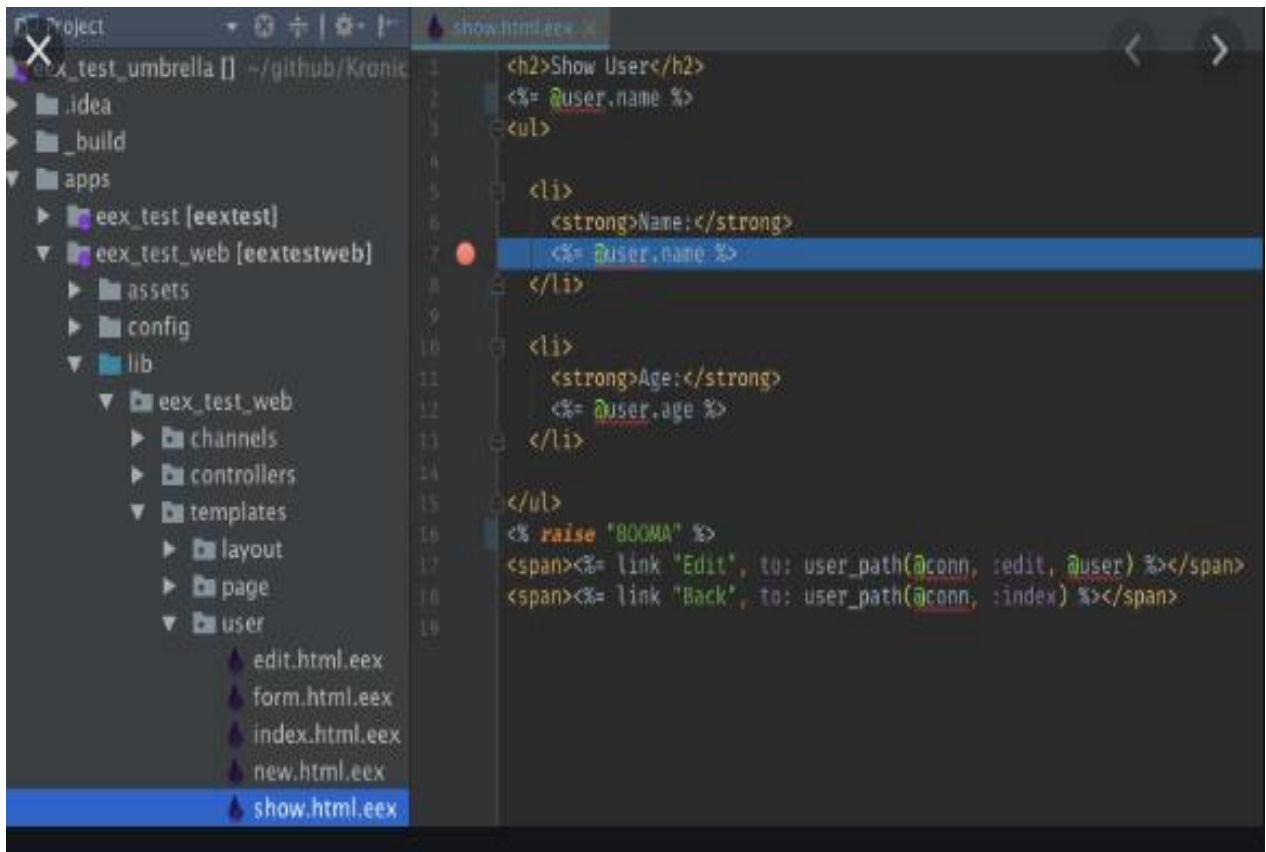


Fig no: 3.20 Programming Page

Running and Debugging

Running a python code comprises of two modes: running a script and debugging the script. This chapter focusses on debugging the Python script using PyCharm.

Steps Involved

The steps for debugging the Python project are as explained below –

Step 1

In the first process, Start with **debugging the Python project**.

Step 2

Now, Windows firewall asks permission for debugging the Python project, as the procedure involves **line-by-line compilation and execution**. This is a standard security measure to ensure that no unauthorized applications are executing code on your system.

You may be prompted to allow PyCharm to communicate through the firewall. Click **Allow Access** to proceed with debugging. If you don't grant this permission, debugging may not work as expected, and the IDE will be unable to execute the program in the interactive debugging environment.

Step 3

The debugging console is created in the PyCharm editor as shown below, which executes the output line by line. When the program is run in debug mode, PyCharm pauses at each breakpoint, allowing you to **inspect variables, evaluate expressions, and track the flow of execution step by step**.

This allows developers to carefully analyze the program's behavior and identify issues or inefficiencies. You can use the **Step Over** (F8) and **Step Into** (F7) buttons to control the execution flow and examine the program's logic at each stage. The console provides valuable feedback and error messages that help in diagnosing and resolving bugs effectively.

3.3 OUTPUT DESIGN

3.3.1 Face Detection

Face detection is a computer vision technique that involves identifying and locating human faces within images or video frames. In the context of the proposed **three-level password authentication** system for an ATM, face detection plays a crucial role in the third authentication level. The system utilizes **OpenCV**, a popular computer vision library in Python, to capture and **analyze facial features**.

Upon initiating a transaction, the ATM's **camera captures the user's face**, and the face detection module processes the image to identify the presence and location of a face. This module employs **pre-trained machine learning** models to recognize facial patterns, distinguishing them from the background.

Once the face is successfully detected, the system proceeds to the face recognition stage, verifying the user's identity by **comparing the captured facial features** with the **stored facial data**. The accuracy and efficiency of the face detection process are pivotal in ensuring the system's overall reliability and security, offering users a seamless and advanced method of authentication.

Face detection is a fundamental aspect of computer vision that involves the identification and localization of faces within visual data, such as images or video frames. In the context of the three-level password authentication system proposed for an ATM, the face detection module plays a pivotal role in the third layer of authentication.

Leveraging the OpenCV library, the system utilizes algorithms and pre-trained models to analyze the images captured by the ATM's camera in real-time. These models can **identify facial features**, distinguishing them from the background, and accurately locate the position of the face within the frame.

The face detection process initiates when a user begins a transaction, prompting the ATM's camera to capture the user's face. This captured image undergoes analysis, and if a face is successfully detected, the system proceeds to the subsequent stage of face recognition.

The effectiveness of the face detection module is crucial for ensuring the accuracy of the authentication process, as an erroneous detection could lead to unauthorized access or denial of service. By incorporating **advanced face detection capabilities**, the ATM system enhances security by adding a sophisticated layer of identity verification.

The **seamless integration of face detection** within the authentication workflow not only contributes to the overall robustness of the system but also ensures a **user-friendly** and efficient experience for **individuals interacting** with the ATM.

3.3.2 Contour Mapping

Contour mapping meticulously captures the intricate details of the Earth's surface, offering a comprehensive **understanding** of its **topography**. Similarly, the facial recognition technology employed in the project meticulously **analyzes** and **records** the **unique features of individuals' faces**, creating a digital representation of their identity.

In contour mapping, the contour lines provide a visual guide, allowing users to navigate through complex terrains with ease. Likewise, the facial recognition system acts as a digital guide, ensuring secure navigation through the **online transaction process** by **authenticating users' identities** with precision and reliability.

Contour mapping requires accuracy to depict the landscape faithfully, the facial recognition technology must **accurately identify individuals** to **safeguard online transactions** effectively. Both processes rely on advanced algorithms and rigorous testing to achieve high levels of accuracy and reliability.

In essence, just as contour mapping facilitates understanding and navigation of physical landscapes, the project's facial recognition technology facilitates secure navigation of the digital landscape, enhancing convenience and safety in online transactions.

3.3.3 SMPT Protocol

SMTP is an application layer protocol. The client who wants to **send the mail** opens a **TCP connection to the SMTP server** and then **sends the mail across the connection**. The SMTP server is an always-on listening mode. As soon as it listens for a TCP connection from any client, the SMTP process **initiates a connection through port 25**. After successfully establishing a TCP connection the client process sends the mail instantly.

SMTP Protocol

The SMTP model is of two types:

- End-to-End Method
- Store-and-Forward Method

The **end-to-end model** is used to **communicate between different organizations** whereas the store and forward method is used within an organization. An SMTP client who wants to send the mail will contact the destination's host SMTP directly, to send the mail to the destination. The SMTP server will keep the mail to itself until it is successfully copied to the receiver's SMTP.

The client SMTP is the one that initiates the session so let us call it the **client-SMTP** and the **server SMTP** is the one that responds to the session request so let us call it receiver-SMTP. The client-SMTP will start the session and the receiver SMTP will **respond to the request**.

Model of SMTP System

In the SMTP model user deals with the **user agent (UA)**, for example, **Microsoft Outlook, Netscape, Mozilla**, etc. To **exchange the mail** using **TCP**, **MTA** is used. The user sending the mail doesn't have to deal with MTA as it is the responsibility of the system admin to set up a local MTA.

The **MTA maintains a small queue of mail** so that it can schedule repeat delivery of mail in case the receiver is not available. The MTA delivers the mail to the mailboxes and the information can later be downloaded by the user agents.

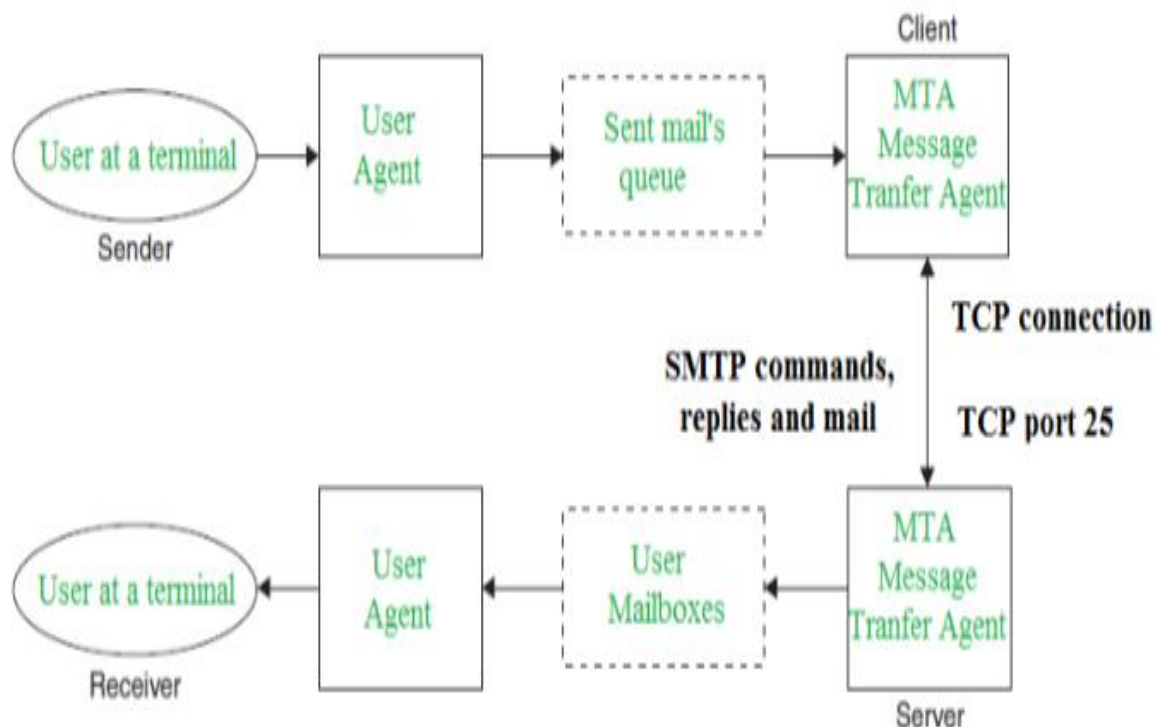


Fig no: 3.25 SMPT Protocol

Components of SMTP

- **Mail User Agent (MUA):** It is a computer application that helps you in **sending and retrieving mail**. It is responsible for creating email messages for transfer to the mail transfer agent (MTA).
- **Mail Submission Agent (MSA):** It is a computer program that **receives mail** from a Mail User Agent(MUA) and interacts with the Mail Transfer Agent(MTA) for the transfer of the mail.
- **Mail Transfer Agent (MTA):** It is software that has the work to **transfer mail** from one system to another with the help of SMTP.
- **Mail Delivery Agent (MDA):** A mail Delivery agent or **Local Delivery Agent** is basically a system that helps in the delivery of mail to the local system.

Working of SMTP

Below mentioned are the steps of the working of SMTP [Simple Mail Transfer Protocol].

- **Communication between the sender and the receiver:** The sender's user agent prepares the message and sends it to the MTA. The MTA's responsibility is to transfer the mail across the network to the receiver's MTA. To send mail, a system must have a client MTA, and to receive mail, a system must have a server MTA.
- **Sending Emails:** Mail is sent by a series of **request and response messages** between the client and the server. The message which is sent across consists of a header and a body. A **null line** is used to **terminate the mail header** and everything after the null line is considered the body of the message, which is a **sequence of ASCII characters**. The message body contains the actual information read by the receipt.
- **Receiving Emails:** The user agent on the **server-side checks the mailboxes** at a particular time of intervals. If any information is received, it informs the user about the mail. When the user tries to read the mail it **displays a list of emails** with a **short description** of each mail in the mailbox. By selecting any of the mail users can view its contents on the terminal.

Advantages of SMTP

- If necessary, the users can have a **dedicated server**.
- It allows for **bulk mailing**.
- **Low cost** and wide coverage area.
- Offer choices for email tracking.
- Reliable and **prompt email delivery**.

Disadvantages of SMTP

- SMTP's common port can be blocked by several firewalls.
- SMTP security is a **bigger problem**.
- Its simplicity **restricts how useful** it can be.
- Just **7-bit ASCII characters** can be used.
- If a message is longer than a certain length, SMTP servers may **reject the entire message**.

3.4 CODE DESIGN

PyCharm is a study and characteristic-packed IDE that many Python builders swear with the aid of, it is not without its negative aspects. These limitations consist of useful resource consumption, a getting-to-know curve, price for the Professional Edition, slower startup instances, and constrained language support. It's critical for builders to not forget their precise desires and options while selecting an IDE, as the "best" IDE can vary depending on the context and individual requirements.

The code design for the ATM security system using facial recognition follows a modular approach to ensure efficiency, security, and maintainability. The Main Program (**main.py**) serves as the entry point, managing workflow and user interactions while calling authentication and transaction modules. The Face Detection Module (**face_detection.py**) captures and preprocesses facial images for **feature extraction**, ensuring **accurate recognition**. The Feature Extraction Module (**feature_extraction.py**) extracts key facial landmarks and converts them into **machine-readable data to facilitate authentication**. The **Real-Time Verification Module (verification.py)** matches captured facial data with stored images, implementing authentication logic to grant or deny ATM access.

The **Database Management Module (database.py)** securely stores user credentials, facial images, and transaction logs, ensuring encrypted data handling and retrieval. **PIN Authentication (pin_auth.py)** serves as a secondary verification method, reinforcing a multi-layered security system. The Transaction Handling Module (**transaction.py**) manages core financial operations, including withdrawals, deposits, and balance inquiries, while maintaining a secure transaction history. The **Email Alert System (email_alert.py)** enhances security by notifying users of unauthorized access attempts and successful transactions in real time.

The **User Interface Module (ui.py)** provides an interactive and user-friendly ATM experience, displaying authentication and transaction options clearly. The **Configuration Settings Module (config.py)** defines security policies, authentication thresholds, and customization parameters for adaptability. The **Error Handling and Logging Module**

(error_log.py) tracks authentication failures, system warnings, and potential breaches, aiding in debugging and security reviews. Lastly, the **Testing and Evaluation Module (test_suite.py)** ensures system reliability through automated tests that validate authentication accuracy, performance, and overall security.

To provide seamless integration, the ATM security system is designed with an efficient communication flow between its components. The main program acts as a **central hub, directing user inputs** and calling relevant modules when needed. During a transaction, the face detection module is first triggered to capture the user's facial image, which is then processed by the **feature extraction module** to **identify key facial landmarks**. Once the extracted features are processed, the **real-time verification module matches** them **against the stored database records**. If the **authentication is successful**, the user proceeds to the next step of **entering their PIN**, where the **system validates** the provided **credentials** before allowing financial operations.

The database management module plays a crucial role in maintaining security and accuracy by **securely storing encrypted user credentials** and **transaction logs**. It ensures that the **stored facial images** are accessed only when **authentication is required, preventing unauthorized access**. In addition, the PIN authentication module provides an extra layer of security, ensuring that even if someone bypasses facial recognition, the system requires an additional validation step before granting access to financial operations.

The transaction handling module streamlines financial interactions by **managing user requests**, including **cash withdrawals, deposits, and balance inquiries**. This module ensures that transactions are executed securely while maintaining logs for future reference. In case of suspicious activities or **authentication failures**, the **email alert system** promptly **notifies users by sending alerts** about failed login attempts or unauthorized access to their registered email. The error handling and logging module ensures system stability by capturing system errors, security breaches, and transaction failures. By maintaining detailed logs, this module helps developers **identify vulnerabilities, debug system errors, and improve system performance**.

3.5 DATABASE DESIGN

The database design for the enhanced ATM security system integrates essential components to ensure **robust user authentication**, transaction management, and system security. The core tables include Users, Facial Data, Transactions, and Alerts. The Users table stores basic user details such as **UserID (primary key)**, **Name**, **Email**, **Phone**, and **encrypted PIN**, ensuring each user is uniquely identified.

The Facial Data table contains biometric information, including **FacialDataID (primary key)**, **UserID (foreign key)**, **FacialImage (encoded format)**, and **LandmarkCoordinates (key facial points)**, which are critical for accurate facial recognition.

The Transactions table logs all ATM activities with fields such as **TransactionID (primary key)**, **UserID (foreign key)**, **TransactionDateTime**, **TransactionAmount**, and **Status (e.g., Success or Failed)**, enabling detailed traceability. Unauthorized access attempts are managed in the Alerts table, which stores **AlertID (primary key)**, **UserID (foreign key)**, **Timestamp**, **CapturedImage (unauthorized user image)**, and **AlertStatus (e.g., Sent or Resolved)**, facilitating real-time notification and response.

Additionally, an AuditLogs table records system events, including **LogID**, **EventDescription**, **EventTimestamp**, and **optional UserID**, for **monitoring** and **troubleshooting**. Relationships between tables are maintained through foreign keys, ensuring referential integrity, with the Users table serving as the central link to related data. Security measures include encryption of sensitive information like PINs and secure storage of facial data to prevent unauthorized access.

Automated email alerts notify users of unauthorized attempts, adding an extra layer of protection. This comprehensive database design ensures seamless functionality, **safeguarding user information** and **financial transactions** while supporting the multi-layered security approach of the ATM system. The database design for the ATM security system with facial recognition is structured to ensure **secure user authentication**, **transaction integrity**, and

real-time security monitoring. At the core of this design is the Users table, which maintains user-specific data such as **UserID (primary key)**, **FullName**, **Email**, **PhoneNumber**, and **EncryptedPIN**.

This table ensures each user is uniquely identifiable while securing personal data through encryption. To complement this, the FacialRecognition table stores **biometric information**, including **FacialID (primary key)**, **UserID (foreign key)**, **EncodedFacialImage**, and **KeyLandmarks**, which are used for authentication and identity verification.

The Transactions table records financial activities, ensuring accountability and traceability. It contains fields such as TransactionID (primary key), UserID (foreign key), Transaction Type (e.g., Withdrawal, Deposit, Balance Inquiry), **Amount**, **TransactionTimestamp**, and Status (Success or Failed). By maintaining detailed transaction logs, the system can track user activity and detect anomalies.

Additionally, the UnauthorizedAccess table captures failed authentication attempts. It includes AlertID (primary key), UserID (foreign key), AttemptTimestamp, CapturedImage (if available), and AlertStatus (e.g., Sent, Resolved). This table enables real-time monitoring of security breaches and notifies users or administrators accordingly.

A crucial component of this database structure is the SystemLogs table, which records internal system activities, including LogID (primary key), EventType, EventDescription, EventTimestamp, and **RelatedUserID (optional)**. This log assists in auditing, debugging, and **analyzing system performance**. Moreover, the SecuritySettings table maintains configurable **security parameters**, such as **SettingID (primary key)**, **SettingName (e.g., MaxFailedAttempts, LockoutDuration)**, and **CurrentValue**, allowing administrators to adapt security policies dynamically.

The design also incorporates an OTP Verification table, which supports multi-factor authentication. This table consists of **OTPID (primary key)**, **UserID (foreign key)**, **OTPCode**, **ExpirationTime**, and **VerificationStatus**, ensuring that temporary security codes are effectively managed. Furthermore, a CardDetails table is introduced to **store ATM card**

information, with attributes including **CardID (primary key)**, **UserID (foreign key)**, **CardNumber (encrypted)**, **ExpiryDate**, and **CardStatus**. The integration of this table allows a secure link between ATM cards and facial recognition authentication.

To enhance the efficiency of alerts, the Email Notifications table manages automated messages sent to users upon security events or successful transactions. This table includes EmailID (primary key), UserID (foreign key), EmailType, SentTimestamp, and DeliveryStatus, ensuring timely communication.

The database enforces referential integrity through foreign key constraints, linking related tables while preventing orphan records. Sensitive data such as PINs and facial images are securely **encrypted to prevent unauthorized access**.

Optimizing system performance, indexed fields such as UserID and TransactionTimestamp enable **rapid data retrieval**, enhancing the system's responsiveness. Regular backups and redundancy measures ensure data availability and resilience against failures. By integrating these components, the **ATM security system achieves a robust, scalable, and secure database design, supporting seamless authentication, transaction processing, and fraud detection mechanisms**.

3.6 SYSTEM DEVELOPMENT

In this project they are 5 modules:-

- Input Camera Module:
- Preprocessing and Segmentation Module:
- Face Recognition Module:
- Dataset Module:
- Output Module:
- Alert module:

3.6.1 Modules Description

1. Input Camera Module:

This module is responsible for **capturing images of individuals** attempting to access the ATM using an integrated camera.

2. Preprocessing and Segmentation Module:

The preprocessing module processes the captured images to **enhance** their **quality** and **remove** any **noise or distortions**, ensuring optimal performance during facial recognition.

3. Face Recognition Module:

This module facial landmark algorithms to analyze and **recognize** the **unique features** of the segmented facial region, allowing for accurate authentication of users' identities.

4. Dataset Module:

The dataset module manages the **storage and retrieval of facial images** and associated user information in a database. It ensures **efficient access to training data** for the facial recognition model.

5. Alert module

The alert module displays the results of the facial recognition process, indicating whether the user has been **successfully authenticated**. It also outputs transaction details, such as the **amount to be withdrawn** or transferred, upon successful authentication. If an **unauthorized access**, the system triggers an **immediate alert**, capturing the image and storing it in the database. Simultaneously, an **email notification** is **sent to the authorized person**.

Detailed Explanation About The Project Work

Objective: The project aims to enhance the security of online transactions by integrating facial recognition technology alongside traditional security measures like **One-Time Passwords (OTPs)**.

Problem Statement: The project addresses the increasing concerns related to **theft and fraud** associated with carrying **physical payment cards** and **cash**, especially in the context of online transactions.

Approach: The project utilizes a **multi-layered security approach**, starting with OTP authentication followed by facial recognition. This approach ensures robust security while maintaining user convenience.

User Interface Design: The project emphasizes a **user-centric approach** in designing the interface, ensuring that it is intuitive, **easy to navigate**, and aesthetically pleasing. :Clear prompts and instructions guide users through each step of the **transaction process**, **enhancing user experience** and minimizing errors.

Authentication Mechanisms: OTP generation and validation are handled securely within the system, employing cryptographic techniques to **generate unique** and **time-sensitive codes**. This ensures that each transaction is protected by a **dynamically generated password**, adding an extra layer of security against unauthorized access.

Facial Recognition Algorithm: The facial recognition module utilizes state-of-the-art algorithms for facial landmark detection, capturing essential features of the user's face with precision and accuracy. By employing techniques such as **Dlib's 68-point detection**, the system can identify users reliably, even in varying lighting conditions and facial expressions.

Database Encryption: To safeguard sensitive user data, the system employs robust encryption techniques to secure the database of authorized facial features. Encryption algorithms such as AES (Advanced Encryption Standard) are used to protect data at rest, ensuring that unauthorized access to the database is prevented.

Workflow:

- Users initiate a transaction through a **user-friendly interface**.
- They are prompted to input a **unique OTP** for additional authentication.
- After **OTP verification**, users undergo facial recognition authentication.
- **Facial features** are **captured** and compared against stored data in the system's database.
- If the face is recognized as authorized, the transaction is completed securely.
- Technology: **Facial landmark detection algorithms**, such as **Dlib's 68-point detection**, are used for facial recognition. Additionally, OTPs are generated and validated securely.

Database Management: The system maintains a database of authorized users' facial features for quick and efficient verification during transactions.

Testing and Evaluation: Extensive testing of the system demonstrates an average accuracy of 95% for face detection and transaction authorization, ensuring robust security measures.

Benefits:

Enhanced security: Multi-layered authentication mitigates risks associated with theft and fraud.

Convenience: Users can conduct **transactions without** the need for **physical cards** or memorizing multiple passwords.

Seamless user experience: Integration of **facial recognition technology** offers a smooth and intuitive transaction process.

Future Implications: The project sets a precedent for the integration of advanced technologies like facial recognition into **online transaction security systems**, paving the way for future advancements in the field.

Overall Impact: By offering a secure and user-friendly solution, the project contributes to building trust and confidence among users and businesses engaging in online transactions.

Alert system: In a unauthorized access, the system triggers an immediate alert, capturing the image and storing it in the database. Simultaneously, an **email notification** is **sent to the authorized person**.

4. TESTING AND IMPLEMENTATION

Types of Testing

1. Unit Testing

Unit testing plays a critical role in ensuring the reliability and functionality of each component within the project's architecture. For the facial recognition module, unit tests are designed to **validate the accuracy and effectiveness of facial landmark detection algorithms**, such as Dlib's 68-point detection.

Tests are conducted using predefined datasets containing images of varying lighting conditions, angles, and facial expressions to simulate real-world scenarios. Each unit test verifies the module's ability to **accurately detect and extract facial features**, such as **eyes**, **nose**, and **mouth**, from input images. Additionally, the OTP generation and validation module undergoes rigorous unit testing to **confirm the correctness of generated OTPs** and the accuracy of OTP validation procedures.

Test cases are designed to cover various scenarios, including edge cases such as **invalid input parameters** and **expired OTPs**. Throughout the unit testing process, automated test scripts are utilized to streamline test execution and generate comprehensive test reports, enabling developers to identify and address any issues promptly. By conducting thorough unit testing, the project ensures that each module functions as intended, contributing to the overall **reliability and security** of the **online transaction system**. Unit testing is a fundamental aspect of ensuring the reliability, security, and functionality of the ATM security system utilizing facial recognition technology.

Each core module undergoes extensive testing to validate its correctness and efficiency. The **face detection module** is tested using **multiple datasets** containing diverse lighting conditions, angles, and facial expressions to simulate real-world usage. Unit tests check whether the system can accurately detect a user's face, extract essential features such as **eyes**, **nose**, and **mouth**, and differentiate between authorized and unauthorized users. The system also tests for false positives and negatives to fine-tune accuracy.

The **facial recognition authentication** module undergoes rigorous testing to validate the system's ability to correctly match stored images with real-time captured images. Tests are conducted using various **facial images to check recognition speed and accuracy**, including edge cases where users may attempt to access the system with partial obstructions such as glasses or masks. Additionally, boundary testing ensures that the module does not allow unauthorized access due to image tampering or low-quality facial scans.

For the PIN authentication module, unit testing validates whether encrypted PINs are correctly stored and retrieved. The test cases include **entering valid** and **invalid PINs**, checking the system's response to repeated incorrect attempts, and ensuring the locking mechanism is triggered after multiple failures. Furthermore, unit tests simulate different user input conditions, such as **incorrect PIN format**, **empty fields**, and **tampered PIN entries**, ensuring robust security enforcement.

Transaction handling undergoes validation to check the **correctness of ATM transactions**, such as **deposits**, **withdrawals**, and **balance inquiries**. Test cases evaluate whether the system correctly processes **transactions** and **updates the database accurately**. Unit tests also simulate edge cases such as attempting to withdraw more than the available balance, network failures during transactions, and handling transaction timeouts. The expected outcomes include error messages, balance updates, and confirmation receipts.

The database management module is tested to ensure secure and efficient data storage and retrieval. Unit tests **validate data consistency**, **encryption**, and proper linkage between users, **facial recognition data**, and **transactions**. The system undergoes stress testing by performing simultaneous user transactions to analyze how well it handles concurrent requests without compromising performance. Tests are also conducted to verify database rollback procedures in case of failed transactions.

The email alert module undergoes validation by simulating different security breach scenarios. Tests check whether an email alert is triggered upon unauthorized access attempts and whether users receive real-time notifications for transactions. The test suite includes scenarios where **email delivery** is delayed, ensuring the system can retry sending notifications

if necessary. Additionally, unit tests validate the correctness of **email content**, **recipient addresses**, and timestamps. **System logs** and **error handling mechanisms** are tested to ensure that all significant events, errors, and security breaches are accurately recorded. Unit tests simulate **authentication failures**, **transaction errors**, and **unexpected crashes** to verify that logs capture essential details, such as timestamps, user IDs, and error descriptions. This ensures efficient debugging and audit tracking.

Finally, performance testing ensures that the system operates smoothly under different loads. Stress tests validate how the ATM security system responds to **multiple authentication requests** and transactions within a given timeframe. Automated unit tests assess the **system's responsiveness**, ensuring that authentication and transactions are processed without unnecessary delays. Security tests are also implemented to check for **potential vulnerabilities**, such as **SQL injection** and **unauthorized data access attempts**.

By conducting thorough unit testing across all modules, the ATM security system guarantees a **secure**, **efficient**, and **reliable experience** for users, **reducing potential risks** and ensuring seamless financial transactions. The structured testing approach enhances confidence in the system's ability to safeguard sensitive data and prevent unauthorized access.

2. Functional testing

Functional testing is essential to validate the **end-to-end functionality** of the online transaction security system, ensuring that it meets the specified requirements and user expectations. The functional testing process involves simulating various user interactions and transaction scenarios to assess the **system's performance** and **behavior**. Test cases are designed to cover all aspects of the transaction process, including initiating transactions, entering OTPs, capturing facial data, and completing transactions securely.

During functional testing, testers verify that the system correctly prompts users to **input OTPs** and **validates** them **against expected criteria**, such as uniqueness and expiration time. Additionally, testers evaluate the accuracy and reliability of the facial recognition module by **verifying** its **ability to detect** and **authenticate users' faces accurately** in different environmental conditions and facial expressions.

Functional testing also encompasses error handling scenarios, ensuring that the system responds appropriately to invalid inputs, timeouts, and other exceptional conditions. Comprehensive test scenarios are executed systematically, covering both normal and edge cases to identify any deviations from expected behavior. By conducting functional testing, the project ensures that the online transaction security system functions seamlessly and reliably, providing users with a secure and intuitive transaction experience.

In the context of the project, functional testing encompasses various aspects of the transaction process. Testers simulate the **initiation of transactions, inputting OTPs, and capturing facial data to verify the system's behavior** at each step. They ensure that users are prompted **correctly to enter OTPs**, and the system validates them accurately against predefined criteria such as uniqueness and expiration time. Moreover, testers evaluate the effectiveness of the facial recognition module by assessing its **ability to detect and authenticate users' faces** accurately across different lighting **conditions, angles, and facial expressions**.

Furthermore, functional testing evaluates the system's error handling capabilities. Testers deliberately introduce invalid inputs, timeouts, and other exceptional conditions to assess how the system responds in such situations. They **verify** that appropriate **error messages** are displayed, and the system gracefully handles unexpected scenarios without compromising security or usability.

By conducting thorough functional testing, the project aims to identify any deviations from expected behavior and address them proactively. This ensures that the **online transaction security system functions** seamlessly and reliably, providing users with a secure and hassle-free transaction experience. Additionally, functional testing helps validate that the system meets the requirements outlined in the project scope, contributing to the overall success and satisfaction of stakeholders.

3. System Testing.

System Testing is a crucial phase in the **development lifecycle** of any **software project**, ensuring that the system meets the specified **requirements** and **functions** as expected. In the case of the proposed online transaction security system utilizing facial recognition technology, rigorous testing procedures were employed to validate its effectiveness and reliability.

The testing process began with unit testing, where individual components such as **OTP verification**, facial landmark detection using **Dlib's 68-point algorithm**, and database management were examined in isolation to ensure their correctness and functionality. This was followed by integration testing, where these components were combined and tested as a whole to verify their interactions and compatibility.

Once the system components were integrated, various scenarios were simulated to evaluate its performance under different conditions. This included testing the system's ability to accurately detect faces in various **lighting conditions**, **angles**, and **facial expressions**, as well as its response to different input scenarios during **OTP verification**.

Furthermore, extensive testing was conducted to assess the system's robustness against **potential security threats** such as **spoofing attacks** or unauthorized access attempts. This involved attempting to bypass the facial recognition system using photographs or videos, as well as testing the system's ability to distinguish between genuine users and impostors. Throughout the testing process, metrics such as accuracy, speed, and reliability were continuously monitored and evaluated. The system demonstrated an **average accuracy of 95%** for **face detection** and **transaction authorization**, meeting the stringent security requirements for online transactions.

4. Integration Testing

Integration testing is a **critical phase** in the development process, ensuring that individual components of a system work together seamlessly to achieve the desired functionality. In the case of the online transaction security system utilizing **facial recognition technology**, integration testing involved combining various modules such as **OTP**

verification, facial landmark detection using Dlib's 68-point algorithm, database management, and **user interface component**. During integration testing, the focus was on **verifying the interactions between these components** and assessing their compatibility. For example, **OTP verification functionality** was integrated with the user interface to ensure that users could input and submit their OTPs successfully. Similarly, facial recognition functionality was integrated with the camera interface to capture and **analyze user faces** during **transaction authentication**.

Test cases were designed to cover different scenarios and edge cases, including **successful transaction flows** as well as error handling scenarios. For instance, tests were conducted to ensure that the system correctly handled cases where the OTP entered by the user was invalid or expired, prompting appropriate error messages and actions. Integration testing involved validating the data flow between components and ensuring that information was accurately passed between them. This included verifying the correct storage and retrieval of authorized user faces in the database, as well as the synchronization of data between the user interface and backend components. Throughout the integration testing process, both **functional and non-functional aspects of the system** were evaluated. This included assessing the system's responsiveness, reliability, and performance under varying conditions.

5. Performance testing

Performance testing is essential to evaluate the system's responsiveness, scalability, and reliability under different workloads and conditions. In the context of the online transaction security system utilizing facial recognition technology, **performance testing focused** on assessing various aspects such as **response time, throughput, and resource utilization**.

One key aspect of performance testing was evaluating the **system's response time** during **transaction authentication**. This involved measuring the time taken for the system to capture the user's face, process it using the **facial landmark detection algorithm**, compare it with **authorized faces in the database**, and complete the transaction authentication process. By conducting tests with varying numbers of concurrent transactions and different levels of

system load, the system's ability to maintain acceptable response times under peak usage scenarios was evaluated. Additionally, performance testing involved assessing the system's throughput, which refers to the number of transactions the system can handle within a given timeframe. This was done by subjecting the system to a high volume of transaction requests and measuring its ability to process them efficiently without degradation in performance.

Resource utilization was another critical aspect of performance testing, focusing on monitoring the **system's consumption of CPU, memory**, and other resources under varying workloads. This helped identify potential bottlenecks or resource constraints that could affect the system's overall performance and scalability.

6. Acceptance Testing

Acceptance testing is the final phase of testing before deploying the system to production, **ensuring that it meets the requirements and expectations** of the end-users. In the context of the online transaction security system utilizing facial recognition technology, acceptance testing focused on validating the **system's functionality, usability**, and overall **user experience**. During acceptance testing, real-world scenarios were simulated to evaluate how well the system performs in typical usage situations. This included testing various aspects such as initiating transactions, **entering OTPs, capturing and verifying user faces**, and **completing transaction authentication processes**.

End-users, including both **stakeholders and potential customers**, were involved in acceptance testing to **provide feedback on the system's usability** and effectiveness. They were tasked with performing common transactions, such as booking hotels, purchasing goods online, or paying bills, while using the system's security features.

Feedback from end-users was collected and **analyzed to identify any issues**, usability concerns, or areas for improvement. This feedback was then used to make necessary adjustments and refinements to the system to ensure that it meets the expectations and requirements of its intended users. In addition to functionality and usability, acceptance testing also focused on ensuring that the system meets any regulatory or compliance **requirements related to online transaction security**. This involved verifying that the

system adheres to industry standards and guidelines for **data protection, privacy, and security**.

Build in test plan

The test plan for the online transaction security system encompasses comprehensive testing procedures to ensure its **functionality, performance, usability**, and compliance. Functional testing will validate the OTP generation, facial recognition accuracy, and database management functionalities. Performance testing will measure system response time, **throughput**, and **resource utilization** under varying loads.

Usability testing will gather **user feedback on interface usability, error handling**, and overall user experience. Compliance testing will verify adherence to regulatory standards, **resilience against security threats**, and **data protection measures**. Through meticulous execution of these testing phases, any potential issues or shortcomings will be identified and addressed, ensuring the system's reliability and effectiveness in securing online transactions.

Implementation

The implementation of the online transaction security system involves the integration of various components, including **OTP generation, facial recognition technology, database management**, and **user interface design**. Firstly, the system architecture will be designed to accommodate these components, ensuring modularity and scalability. OTP generation functionality will be implemented using secure algorithms to generate unique codes for transaction verification. Facial recognition technology, leveraging Dlib's 68-point detection algorithm, will be integrated to authenticate users' identities through facial landmarks.

Database management will involve storing and managing authorized user data securely, ensuring efficient retrieval and updating processes. The user interface will be developed to provide a **seamless and intuitive experience**, guiding users through the transaction process while incorporating error handling mechanisms. Continuous testing and refinement will accompany the implementation process to ensure the **system's functionality, security, and usability meet** the required standards. Upon successful implementation, the system will provide a robust and secure platform for online transactions, enhancing user trust and confidence in their financial interactions.

5. CONCLUSION

In conclusion, the "ATM Security System based on Face Recognition, PIN, and OTP" a comprehensive and innovative approach to **enhance** the **security** of **ATM transactions**. By integrating cutting-edge **biometric technology**, **traditional PIN authentication**, and **dynamic OTP verification**, the system provides a **multi-layered security framework**. The **unique feature of peer verification** by legitimate users further **adds** an **extra layer of trust** and **reliability**. Continuous model training ensures adaptability and accuracy over time.

However, ethical considerations and **user safety** must be paramount, especially in the deployment of incapacitating substances. The proposed system not only addresses the limitations of traditional ATM security methods but also promotes user convenience by reducing reliance on physical cards. The combination of **advanced biometrics** and **dynamic verification** mechanisms positions this project at the forefront of secure and **user-friendly ATM transactions** in the era of digitalization.

6. BIBLIOGRAPY

Books

- Biometric Technology: Authentication, Security, and Privacy by Ravi Das
- Handbook of Smart Antennas for RFID Systems by Nemai Chandra Karmakar
- Biometric Identification, Law, and Ethics by Marcus Smith
- Artificial Intelligence in Banking and Financial Services by Tony Boobier
- ATM Security: Risk Assessment and Strategies by Rajat Kanti Baisya

Articles

- Cardless ATM Transactions: Security Challenges and Solutions by IEEE Xplore
- Facial Recognition Technology for Secure Banking Transactions by Springer
- Enhancing ATM Security Using Multi-Factor Biometric Authentication by ScienceDirect
- A Review on Cardless ATM Systems and their Security Measures by ResearchGate
- AI-Powered Face Recognition in Banking: Benefits & Risks by Harvard Business Review

Websites

- <https://ieeexplore.ieee.org/document/9266077>
- <https://www.finextra.com>
- <https://biometricupdate.com>
- <https://www.nist.gov/programs-projects/biometric-research>
- <https://www.atmmarketplace.com>

7. APPENDICES

A. DATA FLOW DIAGRAM

The data flow diagram illustrates the **flow of data** within the **online transaction** security system, providing insight into **how information** is processed and **exchanged between various components**. At the core of the diagram is the user interface, where users initiate transactions and interact with the system. When a transaction is initiated, the system prompts the user to enter a **one-time password (OTP)** for verification.

This input is then sent to the **OTP verification component**, where it undergoes validation against a generated OTP. Upon successful validation, the system activates the camera to capture the user's face. The captured image is then processed using facial recognition technology, employing **Dlib's 68-point detection algorithm** to **identify facial landmarks** and compare them with authorized faces stored in the database.

If the user's face is recognized as authorized, the **transaction is authenticated**, and the system proceeds to complete the transaction. Additionally, the **system stores the authorized user's face** in the database for future reference. This data flow ensures a secure and seamless transaction process, with **robust authentication** mechanisms in place to **safeguard against unauthorized access** or fraudulent activities.

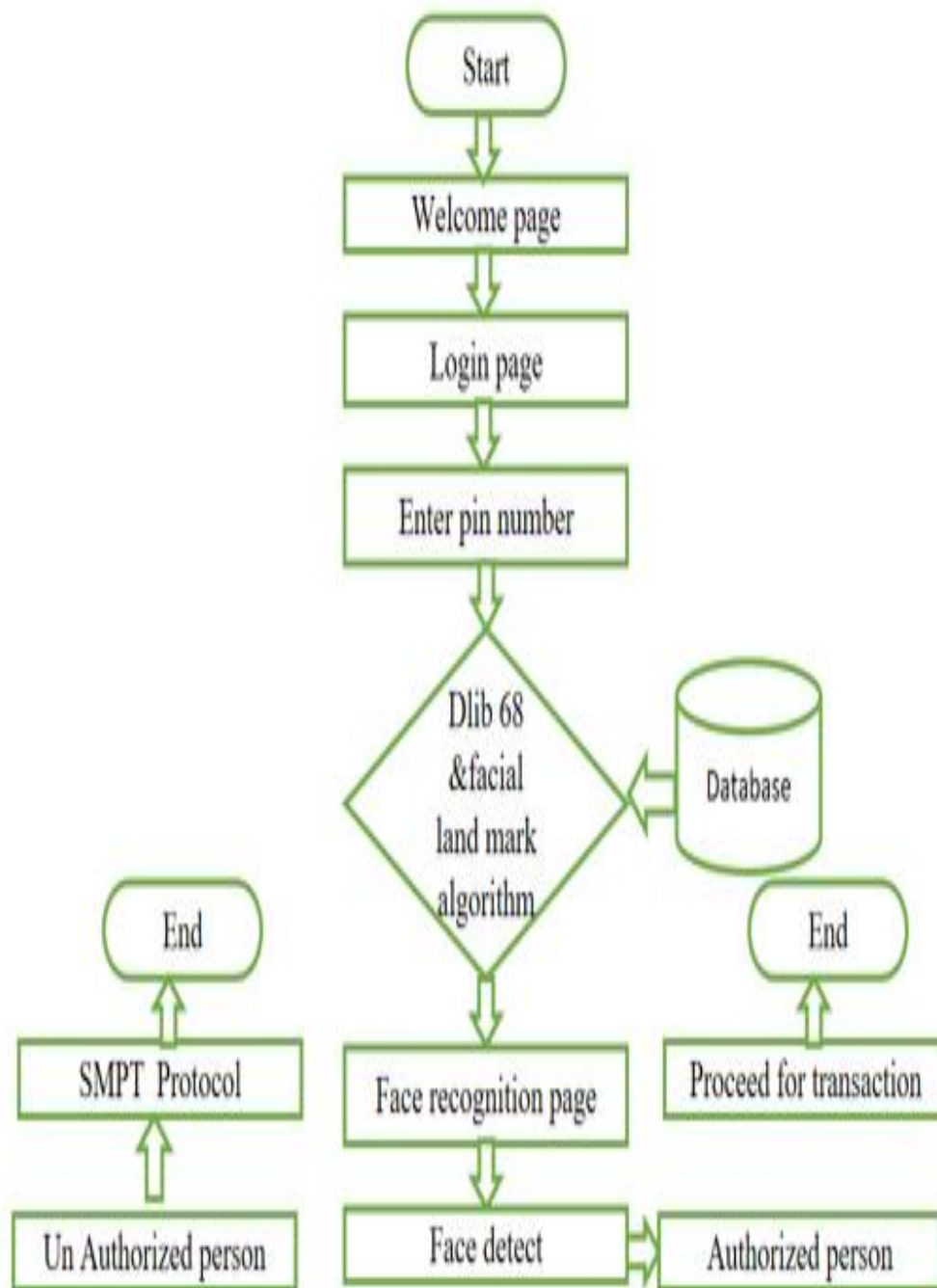


Fig no: 1 Data Flow Diagram

B. SAMPLE CODING

```
from PyQt5 import QtCore, QtGui, QtWidgets

from login import Ui_Form22

class Ui_Form(object):

    def setupUi(self, Form):

        Form.setObjectName("Form")

        Form.resize(801, 618)

        self.label = QtWidgets.QLabel(Form)

        self.label.setGeometry(QtCore.QRect(0, 0, 811, 621))

        self.label.setText("")

        self.label.setPixmap(QtGui.QPixmap("atm.png"))

        self.label.setScaledContents(True)

        self.label.setObjectName("label")

        self.label_2 = QtWidgets.QLabel(Form)

        self.label_2.setGeometry(QtCore.QRect(250, 10, 251, 61))

        font = QtGui.QFont()

        font.setPointSize(35)

        font.setBold(True)

        font.setWeight(75)

        self.label_2.setFont(font)

        self.label_2.setStyleSheet("color: rgb(240,44,44); background-color: rgb(212,242,249);")
```

```
self.label_2.setObjectName("label_2")

self.label_3 = QtWidgets.QLabel(Form)

self.label_3.setGeometry(QtCore.QRect(50, 40, 711, 51))

font = QtGui.QFont()

font.setPointSize(14)

font.setBold(True)

font.setWeight(75)

self.label_3.setFont(font)

self.label_3.setStyleSheet("color: rgb(255, 255, 255);")

self.label_3.setObjectName("label_3")

self.label_4 = QtWidgets.QLabel(Form)

self.label_4.setGeometry(QtCore.QRect(50, 360, 711, 51))

font = QtGui.QFont()

font.setPointSize(14)

font.setBold(True)

font.setWeight(75)

self.label_4.setFont(font)

self.label_4.setStyleSheet("color: rgb(255, 255, 255);")

self.label_4.setObjectName("label_4")

self.label_5 = QtWidgets.QLabel(Form)

self.label_5.setGeometry(QtCore.QRect(50, 300, 711, 51))

font = QtGui.QFont()
```

```
font.setPointSize(14)

font.setBold(True)

font.setWeight(75)

self.label_5.setFont(font)

self.label_5.setStyleSheet("color: rgb(255, 255, 255);")

self.label_5.setObjectName("label_5")

self.label_6 = QtWidgets.QLabel(Form)

self.label_6.setGeometry(QtCore.QRect(50, 250, 711, 51))

font = QtGui.QFont()

font.setPointSize(14)

font.setBold(True)

font.setWeight(75)

self.label_6.setFont(font)

self.label_6.setStyleSheet("color: rgb(255, 255, 255);")

self.label_6.setObjectName("label_6")

self.pushButton = QtWidgets.QPushButton(Form)

self.pushButton.setGeometry(QtCore.QRect(320, 382, 231, 71))

font = QtGui.QFont()

font.setPointSize(19)

font.setBold(True)

font.setWeight(75)
```

```

self.pushButton.setFont(font)

self.pushButton.setStyleSheet("border-radius:30px;\n"
"background-color: rgb(8, 188, 188);")

self.pushButton.setObjectName("pushButton")

self.pushButton.clicked.connect(self.next_page)

self.retranslateUi(Form)

QtCore.QMetaObject.connectSlotsByName(Form)

def retranslateUi(self, Form):
    _translate = QtCore.QCoreApplication.translate
    Form.setWindowTitle(_translate("Form", "Form"))
    self.label_2.setText(_translate("Form", "WELCOME "))
    self.pushButton.setText(_translate("Form", "NEXT"))

def next_page(self):
    print("hello")

self.Form22 = QtWidgets.QMainWindow()

self.Ui = Ui_Form22()

self.Ui.setupUi(self.Form22)

self.Form22.show()

if __name__ == "__main__":
    import sys

    app = QtWidgets.QApplication(sys.argv)

    Form = QtWidgets.QWidget()

    ui = Ui_Form()

```

```

ui.setupUi(Form)

Form.show()

sys.exit(app.exec_())

# -*- coding: utf-8 -*-

# Form implementation generated from reading ui file 'page2.ui'

## Created by: PyQt5 UI code generator 5.15.4

## WARNING: Any manual changes made to this file will be lost when pyuic5 is
# run again. Do not edit this file unless you know what you are doing.

from PyQt5 import QtCore, QtGui, QtWidgets

from face import Ui_Form31

class Ui_Form22(object):

    def setupUi(self, Form22):

        Form22.setObjectName("Form22")

        Form22.resize(745, 591)

        self.label = QtWidgets.QLabel(Form22)

        self.label.setGeometry(QtCore.QRect(0, 0, 751, 601))

        self.label.setText("")

        self.label.setPixmap(QtGui.QPixmap("pin.jpg"))

        self.label.setScaledContents(True)

        self.label.setObjectName("label")

        self.label_2 = QtWidgets.QLabel(Form22)

        self.label_2.setGeometry(QtCore.QRect(300, 10, 100, 51))

        font = QtGui.QFont()

```



```

font.setFamily("Times New Roman")

font.setPointSize(35)

font.setBold(True)

font.setWeight(75)

self.label_2.setFont(font)

self.label_2.setStyleSheet("color: rgb(240,44,44);background-color: rgb(212,242,249);")

self.label_2.setObjectName("label_2")

self.label_3 = QtWidgets.QLabel(Form22)

self.label_3.setGeometry(QtCore.QRect(80, 190, 261, 51))

font = QtGui.QFont()

font.setFamily("Times New Roman")

font.setPointSize(24)

font.setBold(True)

font.setWeight(75)

self.label_3.setFont(font)

self.label_3.setStyleSheet("color: rgb(255,255,255);")

self.label_3.setObjectName("label_3")

self.lineEdit_2 = QtWidgets.QLineEdit(Form22)

self.lineEdit_2.setGeometry(QtCore.QRect(350, 190, 251, 41))

font = QtGui.QFont()

font.setFamily("Times New Roman")

font.setPointSize(14)

self.lineEdit_2.setFont(font)

```

```

self.lineEdit_2.setEchoMode(QtWidgets.QLineEdit.Password)

self.lineEdit_2.setObjectName("lineEdit_2")

self.pushButton = QtWidgets.QPushButton(Form22)

self.pushButton.setGeometry(QtCore.QRect(270, 500, 171, 51))

font = QtGui.QFont()

font.setFamily("Times New Roman")

font.setPointSize(14)

font.setBold(True)

font.setWeight(75)

self.pushButton.setFont(font)

self.pushButton.setStyleSheet("border-radius:20px;\n"
"background-color: rgb(149, 149, 149);")

self.pushButton.setObjectName("pushButton")

self.pushButton.clicked.connect(self.next_page)

self.lineEdit_21 = QtWidgets.QLineEdit(Form22)

self.lineEdit_21.setGeometry(QtCore.QRect(60, 400, 381, 51))

self.lineEdit_21.setStyleSheet("background-color: rgba(0, 0, 0,0);\n"
"border:none;\n"
"border-bottom:2px solid rgba(255,255,255,255);\n"
"color: rgb(255,255, 255);\n"
"padding-bottom:7px;\n")

```

```

"font: 24pt \"MS Shell Dlg 2\";\n"

""

self.lineEdit_21.setObjectName("lineEdit_21")

self.retranslateUi(Form22)

QtCore.QMetaObject.connectSlotsByName(Form22)

def retranslateUi(self, Form22):

    _translate = QtCore.QCoreApplication.translate

    Form22.setWindowTitle(_translate("Form2", "Form"))

    self.label_2.setText(_translate("Form2", "PIN "))

    self.label_3.setText(_translate("Form2", "ENTER ATM PIN:"))

    self.pushButton.setText(_translate("Form2", "VERIFY"))

    def next_page(self):

        n2 = self.lineEdit_2.text()

        if n2 == '1234':

            print("correct PIN number")

            self.lineEdit_21.setText("correct PIN number")

            print("PIN authenticated successfully")

            self.Form31 = QtWidgets.QMainWindow()

            self.ui = Ui_Form31()

            self.ui.setupUi(self.Form31)

            self.Form31.show()

```

```

else:

print("wrong PIN number")

self.lineEdit_21.setText("wrong PIN number")

if __name__ == "__main__":

import sys

app = QtWidgets.QApplication(sys.argv)

Form22 = QtWidgets.QWidget()

ui = Ui_Form22()

ui.setupUi(Form22)

Form22.show()

sys.exit(app.exec_())

import cv2

import mediapipe as mp

import os

import json

import face_recognition

import csv

import time

# Create a directory to save the landmarks

output_dir = "landmarks_output"

os.makedirs(output_dir, exist_ok=True)

```

```

# Create a directory for known faces

known_faces_folder = "faces"

os.makedirs(known_faces_folder, exist_ok=True)

# Initialize Face Mesh

mp_face_mesh = mp.solutions.face_mesh

face_mesh = mp_face_mesh.FaceMesh()

# Function to load known face encodings

def load_known_encodings(known_faces_folder):

    known_face_encodings = []

    known_face_names = []

    for filename in os.listdir(known_faces_folder):

        if filename.endswith((".jpg", ".png")):

            name = os.path.splitext(filename)[0]

            image_path = os.path.join(known_faces_folder, filename)

            face_image = face_recognition.load_image_file(image_path)

            face_encoding = face_recognition.face_encodings(face_image)[0]

            known_face_encodings.append(face_encoding)

            known_face_names.append(name)

    return known_face_encodings, known_face_names

# Load known face encodings and names

known_face_encodings, known_face_names

= load_known_encodings(known_faces_folder)

```

```

# Initialize Video Capture

video_capture = cv2.VideoCapture(0)

# Use the default camera (change to a file or camera index if needed)

while True:

    # Capture frame-by-frame

    ret, frame = video_capture.read()

    if not ret:

        break

    # Convert the frame to RGB for face mesh and face recognition

    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    # Perform face mesh detection

    result = face_mesh.process(rgb_frame)

    # Get image dimensions

    height, width, _ = frame.shape

    # Create a list to store landmarks

    landmarks_list = []

    if result.multi_face_landmarks:

        # Iterate through detected faces

        for facial_landmarks in result.multi_face_landmarks:

            for i in range(0, 468):

                pt1 = facial_landmarks.landmark[i]

                x = int(pt1.x * width)

```

```

y = int(pt1.y * height)

landmarks_list.append((x, y))

# Save the landmarks as a JSON file
output_filename = "landmarks.json"

output_path = os.path.join(output_dir, output_filename)

with open(output_path, "w") as json_file:

    json.dump(landmarks_list, json_file)

# Real-time face recognition

cv2.rectangle(frame, (left, top), (right, bottom), (0, 255, 0), 2)

# Display the name (without confidence) on top of the rectangle
font = cv2.FONT_HERSHEY_DUPLEX

cv2.putText(frame, name, (left + 6, top - 6), font, 2, (0, 255, 0), 2)

# Display the resulting frame

cv2.imshow('Video', frame)

# Break the loop if 'q' is pressed

if cv2.waitKey(1) & 0xFF == ord('q'):

    break

# Release the video capture object and close all windows

video_capture.release()

cv2.destroyAllWindows()

```

C. SAMPLE INPUT AND OUTPUTS

Sample input

In the **online transaction security system**, a sample input scenario involves a user initiating a transaction, such as purchasing goods online. Upon reaching the payment stage, the user is prompted to enter a **one-time password (OTP) received** through their registered mobile device.

The user inputs the **OTP** into the system for **verification**. Subsequently, the system activates the camera to capture the user's face for facial recognition authentication. The captured image undergoes processing using **Dlib's 68-point detection algorithm**, analyzing facial landmarks for comparison with authorized faces stored in the database. If the user's face is successfully recognized as authorized, the transaction proceeds, ensuring a secure and streamlined process for online transactions.

Sample output

Upon successful completion of the facial recognition authentication process in the online transaction security system, the result is a secure and authorized transaction. The system verifies the user's identity by comparing **facial landmarks detected** from the captured image with those stored in the database. If the user's face is recognized as authorized, indicating a match with the stored data, the transaction proceeds seamlessly.

This result ensures that only authorized users can complete transactions, enhancing security and **reducing** the risk of **fraudulent activities**. Additionally, the system may log the transaction details for future reference, providing an audit trail for accountability and record-keeping purposes. Overall, the successful result of the facial recognition authentication reinforces the system's robustness and reliability in safeguarding online transactions against unauthorized access.

SCREENSHOTS



Fig no: 1 Welcome Page



Fig no: 2 ATM Pin Page

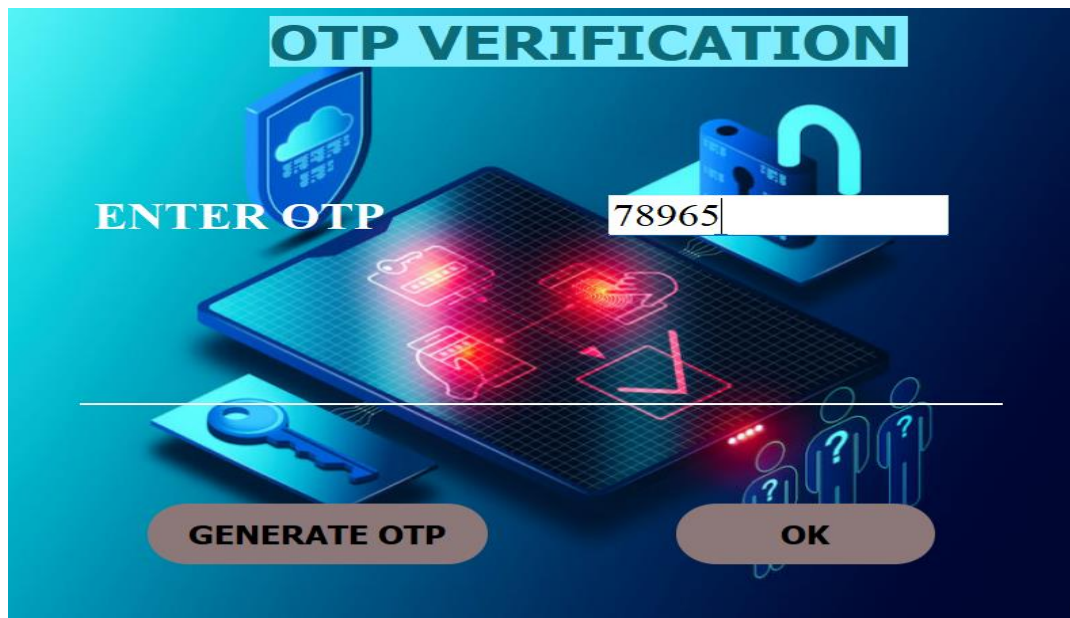


Fig no:3 OTP Verification Page

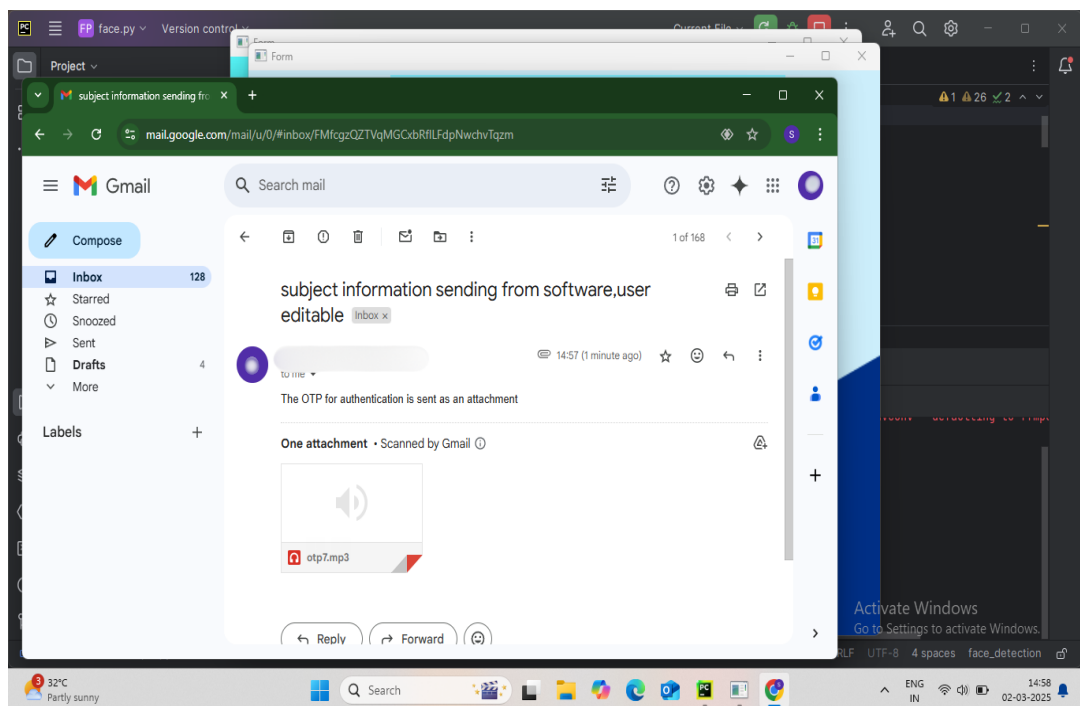


Fig no: 4 OTP via Email Page

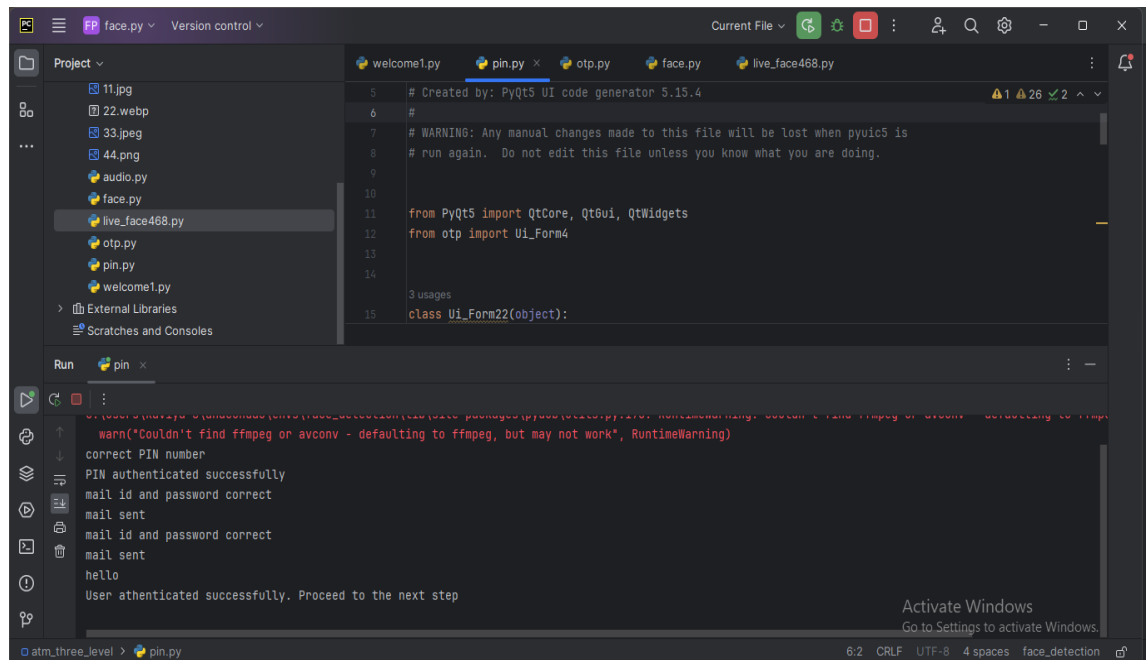


Fig no: 5 Output for Welcome, PIN, OTP Verification and Mail Sent



Fig no: 6 Face Recognition Page

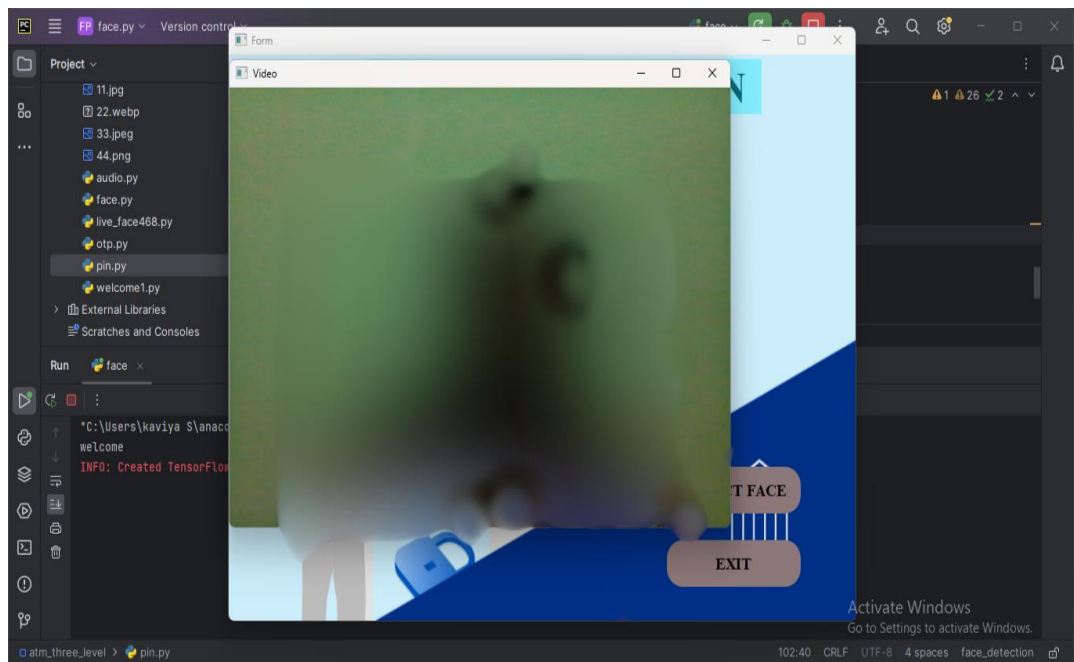


Fig no: 7 Face Detection Page

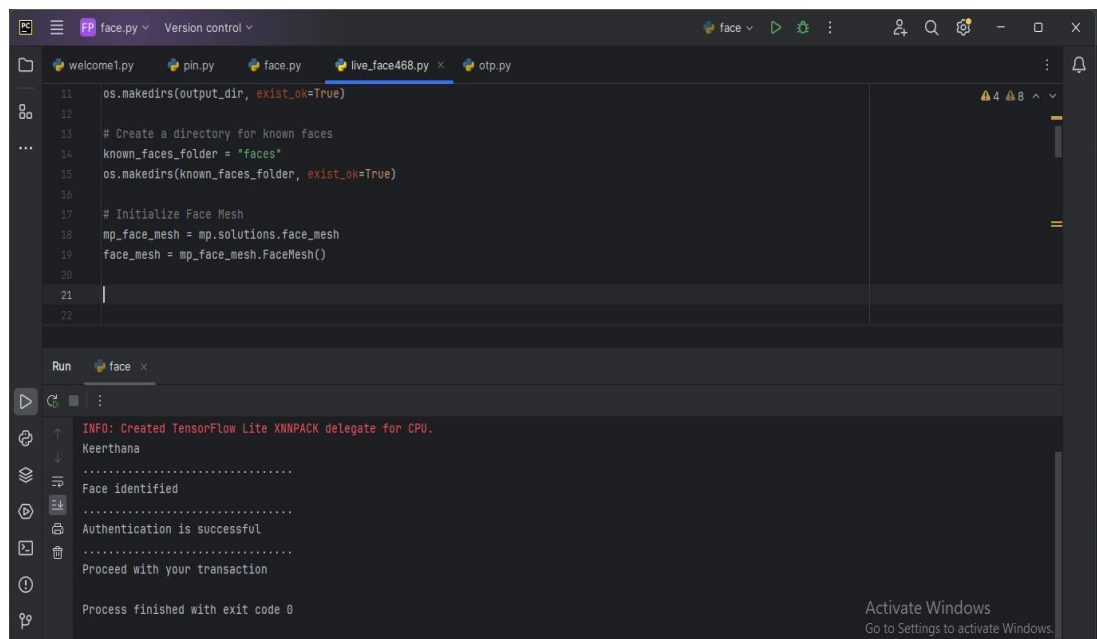


Fig no: 8 Output After Detected the Face